

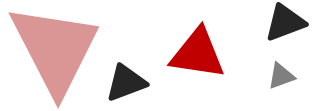
# 单片机

## ——芯火班朋辈辅导

主讲人：张佳豪

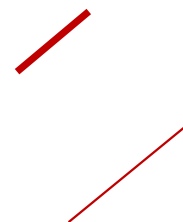
- [ 1 ] **Chap1 基础知识**
- [ 2 ] **Chap2 基本结构**
- [ 3 ] **Chap3 汇编指令系统**
- [ 4 ] **Chap4 汇编程序设计**
- [ 5 ] **Chap7 存储器扩展**

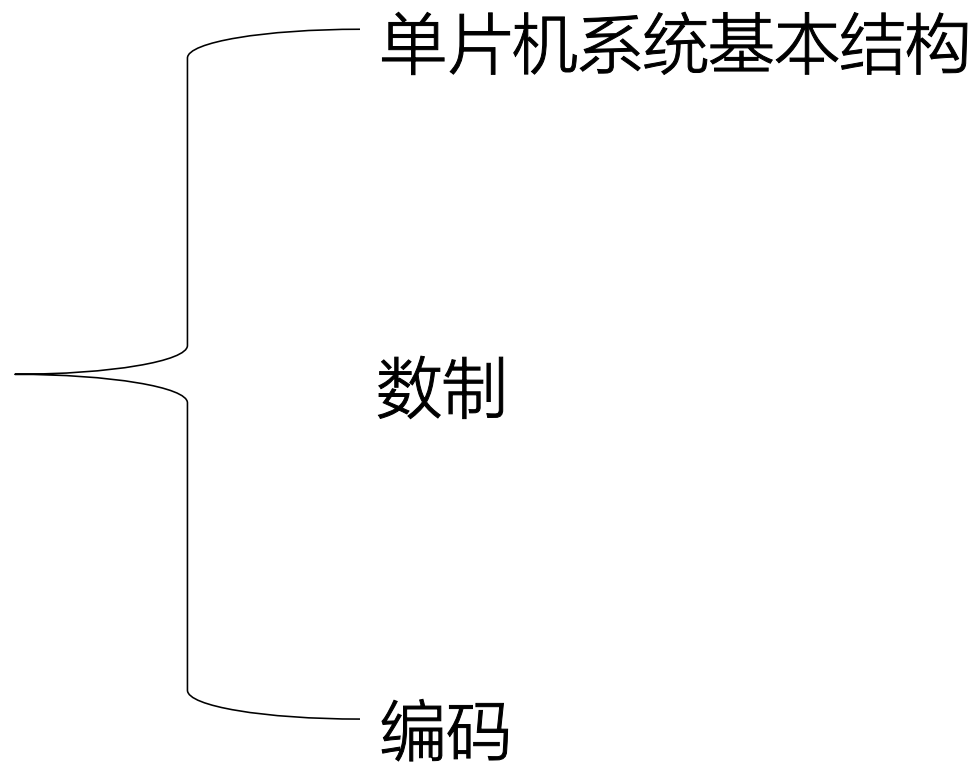
# 1



## Chapter

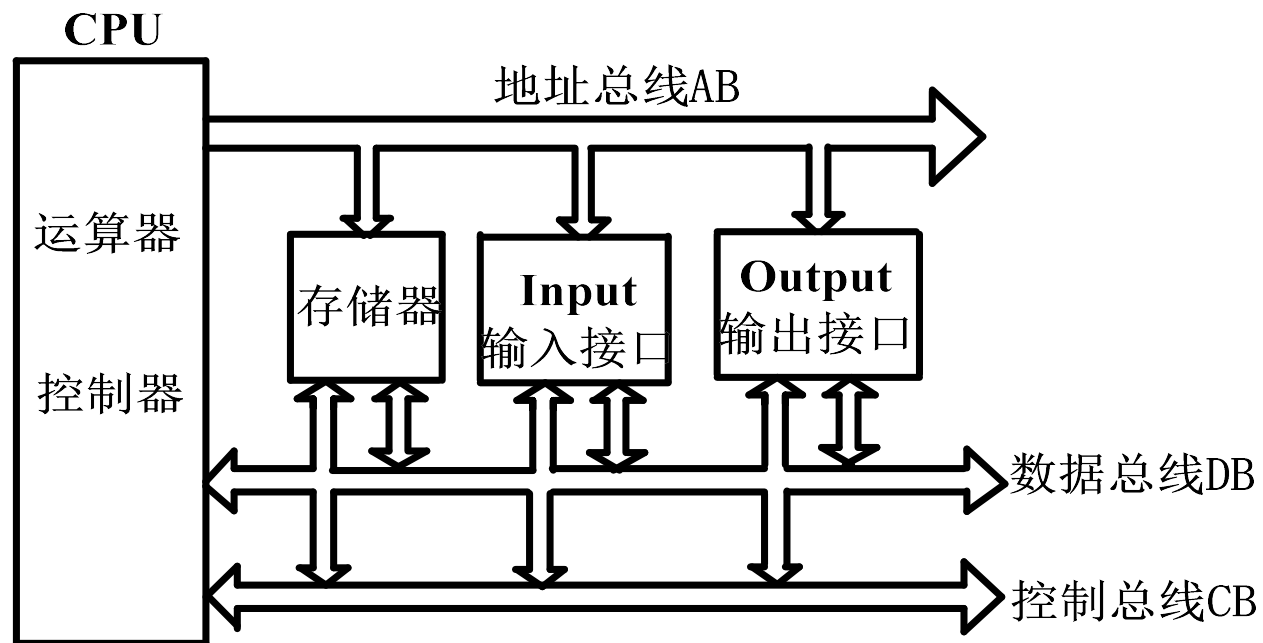
基础知识





## 1 单片机系统基本结构

五大组成部分：运算器、控制器、存储器、输入设备和输出设备



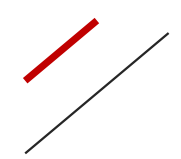


## 1 单片机系统基本结构

### • 单片机最小系统

所谓单片机的**最小系统**，就是指在尽可能少的外部电路的条件下，形成一个可以独立工作的单片机系统。

如8051在加上相应的**复位**和**振荡**电路后就构成了一个最小系统，而对于8031还需要扩展**外部程序存储器**才能构成最小系统。



## 1 数制

### 基数与位权

$$(N)_{10} = \sum_{i=-m}^{n-1} K_i \times 10^i$$

$$(N)_2 = \sum_{i=-m}^{n-1} K_i \times 2^i$$

存储单位 1Byte(字节) = 8Bit

机器数三种表示方法：原码、反码、补码

原码范围： $-(2^{n-1}) \sim +(2^{n-1})$  八位二进制数：-127~127

反码范围： $-(2^{n-1}) \sim +(2^{n-1})$  八位二进制数：-127~127

补码范围： $-(2^n) \sim +(2^{n-1})$  八位二进制数：**-128**~127

(原因：在原码/反码中0被重复编码，补码中0的表示唯一)

在微机中，**普遍采用补码来表示带符号的数**

## [ 1 编码

数制转换——数电复习过不讲

8421BCD码：压缩BCD码：4位2进制数表示10进制数

非压缩BCD码：8位2进制数，高四位0000

（BCD码加法需要用DA指令加6调整；

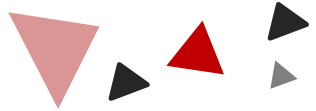
注意BCD码与十六进制数的转换）

ASCII码：0~9的ASCII码：30H~39H；A~Z的ASCII码：41H~5AH

ASCII码最高位为奇偶校验位，非特殊说明考试默认置0



# 2



## Chapter

基本结构





## 2 大纲

结构简介

运算器：**ALU、A、B、PSW**

控制器：**PC、DPTR、SP**

存储器：**ROM、RAM**（通用寄存器、**SFR**）

输入/输出设备：**P0~P3**

复位状态

单片机时序

## 2 结构简介

|    |                                |                  |    |
|----|--------------------------------|------------------|----|
| 1  | P1.0                           | VCC              | 40 |
| 2  | P1.1                           | P0.0/AD0         | 39 |
| 3  | P1.2                           | P0.1/AD1         | 38 |
| 4  | P1.3                           | P0.2/AD2         | 37 |
| 5  | P1.4                           | P0.3/AD3         | 36 |
| 6  | P1.5                           | P0.4/AD4         | 35 |
| 7  | P1.6                           | P0.5/AD5         | 34 |
| 8  | P1.7                           | P0.6/AD6         | 33 |
| 9  | RST/VPD                        | P0.7/AD7         | 32 |
| 10 | P3.0/RXD                       | E $\bar{A}$ /VPP | 31 |
| 11 | P3.1/TXD                       | ALE/PROG         | 30 |
| 12 | P3.2/ $\overline{\text{INT0}}$ | PDEN             | 29 |
| 13 | P3.3/ $\overline{\text{INT1}}$ | P2.7/A15         | 28 |
| 14 | P3.4/T0                        | P2.6/A14         | 27 |
| 15 | P3.5/T1                        | P2.5/A13         | 26 |
| 16 | P3.6/ $\overline{\text{WR}}$   | P2.4/A12         | 25 |
| 17 | P3.7/ $\overline{\text{RD}}$   | P2.3/A11         | 24 |
| 18 | XTAL2                          | P2.2/A10         | 23 |
| 19 | XTAL1                          | P2.1/A9          | 22 |
| 20 | VSS                            | P2.0/A8          | 21 |

- CPU: 单片机的核心, 完成运算和控制功能;
- 内部数据存储器: 256个RAM单元, 存储数据;
- 特殊功能寄存器(SFR), **教材32页表2.5.3**: 用来对片内各个部件进行管理、控制、监视的控制寄存器和状态寄存器, 是特殊的RAM区: 80H~FFH;
- 4KB内部程序存储器: 用于存储程序、原始数据或表格。
- 并行I/O口: 4个8位口(P0~P3), 实现数据的输入、输出;
- 串口: 一个全双工的串口, 实现与其他设备的通信, 如PC机;
- 定时器: 2个16位定时器(51); 3个16位定时器(52);
- 中断: (51): 外部中断2个, 定时/计数中断2个, 串行中断1个; (52): 外部中断2个, 定时/计数中断3个, 串行中断1个
- 1个振荡电路: 时钟源。

89C51=(8位)CPU + 4KBROM + 256BRAM + (2×16)T/C + (4×8)I/O  
+ 1个UART + 5个INT

## 2 结构简介

|    |                         |                      |    |
|----|-------------------------|----------------------|----|
| 1  | P1.0                    | VCC                  | 40 |
| 2  | P1.1                    | P0.0/AD0             | 39 |
| 3  | P1.2                    | P0.1/AD1             | 38 |
| 4  | P1.3                    | P0.2/AD2             | 37 |
| 5  | P1.4                    | P0.3/AD3             | 36 |
| 6  | P1.5                    | P0.4/AD4             | 35 |
| 7  | P1.6                    | P0.5/AD5             | 34 |
| 8  | P1.7                    | P0.6/AD6             | 33 |
| 9  | RST/VPD                 | P0.7/AD7             | 32 |
| 10 | P3.0/RXD                | $\overline{EA}$ /VPP | 31 |
| 11 | P3.1/TXD                | ALE/PROG             | 30 |
| 12 | P3.2/ $\overline{INT0}$ | PDEN                 | 29 |
| 13 | P3.3/ $\overline{INT1}$ | P2.7/A15             | 28 |
| 14 | P3.4/T0                 | P2.6/A14             | 27 |
| 15 | P3.5/T1                 | P2.5/A13             | 26 |
| 16 | P3.6/ $\overline{WR}$   | P2.4/A12             | 25 |
| 17 | P3.7/ $\overline{RD}$   | P2.3/A11             | 24 |
| 18 | XTAL2                   | P2.2/A10             | 23 |
| 19 | XTAL1                   | P2.1/A9              | 22 |
| 20 | VSS                     | P2.0/A8              | 21 |

### 总线组成：

- 地址总线（AB/Address Bus）P0（低八位）P2（高八位）
- 数据总线（DB/Data Bus）P0
- 控制总线（CB/Control Bus）P3口第二功能状态、四根独立控制线RST、EA、ALE、PSEN。

## 2 运算器

运算器的用途：对数据进行算术运算和逻辑操作

运算器的任务：计算缓冲器内容→暂存→修改运行标志

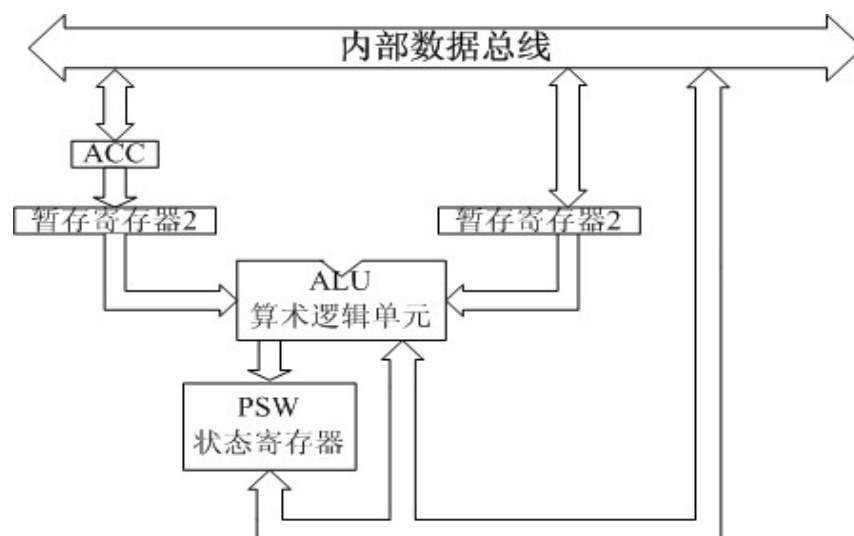
运算器的组成：累加器**ACC**、程序状态字寄存**PSW**、...

### 一、累加器A

- 存放操作数或中间运算结果；
- 是利用率最高的寄存器。

### 二、寄存器B

- 一般与累加器A配合使用，用于乘、除法指令
- 运算前：乘数/除数
- 运算后：存储乘积的高位字节/商的余数



## 2 运算器

### 三、程序状态字寄存器PSW (Program State Word)

| PSW. 7 | PSW. 6 | PSW. 5 | PSW. 4 | PSW. 3 | PSW. 2 | PSW. 1 | PSW. 0 |
|--------|--------|--------|--------|--------|--------|--------|--------|
| CY     | AC     | F0     | RS1    | RS0    | OV     | F1     | P      |
| 位 7    | 位 6    | 位 5    | 位 4    | 位 3    | 位 2    | 位 1    | 位 0    |

**Cy: 进位标志** 加减运算，产生进位/借位，**Cy由硬件置1，否则置0。**

在位操作中常做位累加器（地位相当于累加器A）

**AC: 半进位标志** 加减运算，操作结果低半字节向高半字节进位/借位，硬件置1，否则置0。

**在BCD码调整运算（DA）会用到AC。**

**F0、F1: 用户标志位** 程序设计时可以使用，地位相当于20H.0。

**P: 奇偶标志位** **A中有奇数个1，则P置1，否则置0。**（串口通信奇偶校验）

## 2 运算器

### 三、程序状态字寄存器PSW (Program State Word)

| PSW. 7 | PSW. 6 | PSW. 5 | PSW. 4 | PSW. 3 | PSW. 2 | PSW. 1 | PSW. 0 |
|--------|--------|--------|--------|--------|--------|--------|--------|
| CY     | AC     | F0     | RS1    | RS0    | OV     | F1     | P      |
| 位 7    | 位 6    | 位 5    | 位 4    | 位 3    | 位 2    | 位 1    | 位 0    |

**RS1、RS0:** 工作寄存器组指针——上电复位时，RS1、RS0为0，采用组号0

| RS1 | RS0 | 默认组号 | R0  | R1  | R2  | R3  | R4  | R5  | R6  | R7  |
|-----|-----|------|-----|-----|-----|-----|-----|-----|-----|-----|
| 0   | 0   | 0    | 00H | 01H | 02H | 03H | 04H | 05H | 06H | 07H |
| 0   | 1   | 1    | 08H | 09H | 0AH | 0BH | 0CH | 0DH | 0EH | 0FH |
| 1   | 0   | 2    | 10H | 11H | 12H | 13H | 14H | 15H | 16H | 17H |
| 1   | 1   | 3    | 18H | 19H | 1AH | 1BH | 1CH | 1DH | 1EH | 1FH |

## 2 运算器

### 三、程序状态字寄存器PSW (Program State Word)

| PSW. 7 | PSW. 6 | PSW. 5 | PSW. 4 | PSW. 3 | PSW. 2 | PSW. 1 | PSW. 0 |
|--------|--------|--------|--------|--------|--------|--------|--------|
| CY     | AC     | F0     | RS1    | RS0    | OV     | F1     | P      |
| 位 7    | 位 6    | 位 5    | 位 4    | 位 3    | 位 2    | 位 1    | 位 0    |

**OV:** 溢出标志，有符号数加减/无符号数乘除，有异常结果，OV硬件置1，否则清零。

$$OV = C_{6Y} \oplus C_{7Y}$$

其中，C<sub>6Y</sub>是数值最高位相加进位，C<sub>7Y</sub>是符号位相加进位。

如：00011001 (+25) + 01111101 (+125) = 10010110 (负数)，符号位相加进位0，数值位相加进位1，OV=1，溢出；**Cy=0**

10000111 (-121) + 10001000 (-120) = 00001111 (正数)，符号位相加进1，数值位相加进0，OV=1，溢出；**Cy=1**

—— (注意区分Cy与OV)



## 2 控制器

**控制器的用途：**统一指挥和控制各单元协调工作

**控制器的任务：**从ROM中取出指令→译码→执行指令

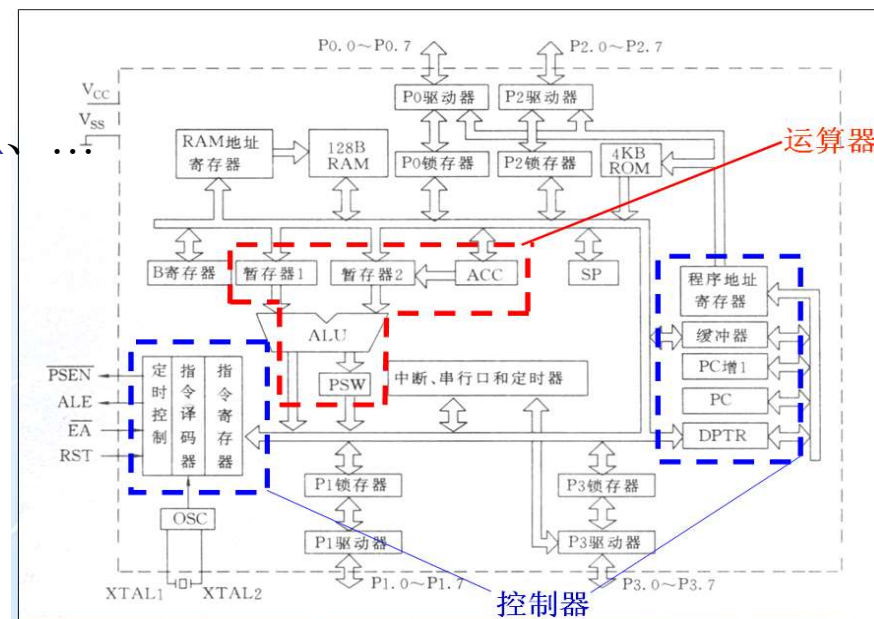
**控制器的组成：**程序计数器**PC**、数据指针寄存器**DPTR**、

### 一、程序计数器PC

- 指向ROM存储单元
- 永远存放着指向下一条指令的16位地址
- 可寻址范围 $2^{16}$
- 顺序执行自动+1，子程序跳转被入栈保护

### 二、数据指针计数器DPTR (DPH+DPL)

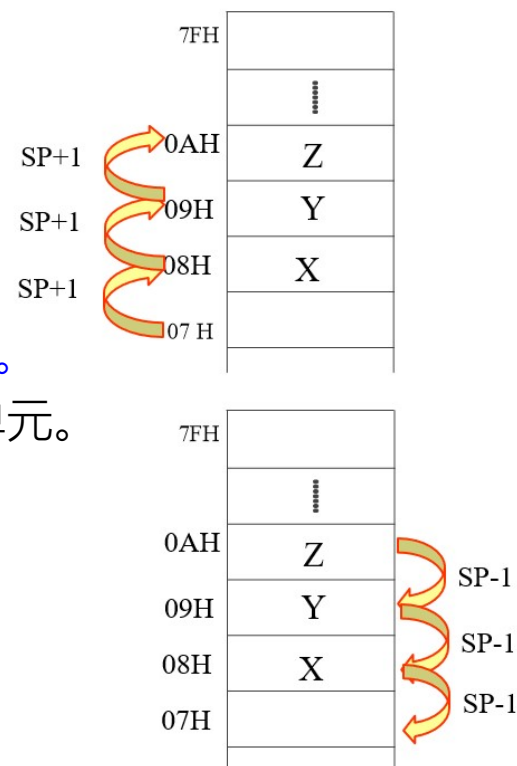
- 指向ROM（变址寻址）/RAM存储单元的地址指针
- 可以变更数据地址
- 可以拆分成DPH与DPL分别赋值



## 2 控制器

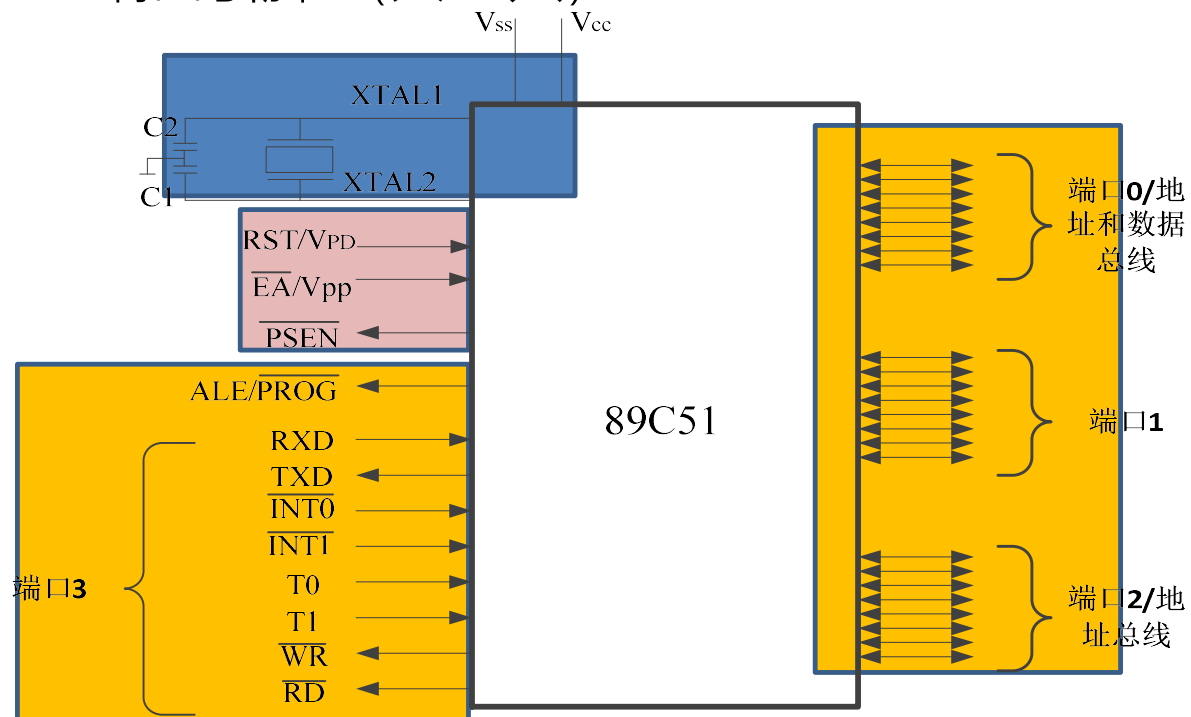
### 三、堆栈指针SP

- MCS-51单片机的堆栈，是在片内RAM中开辟的一个专用区，用来暂时存放数据或存放返回地址，并按照“后进先出”（LIFO）的原则进行操作。
- 栈底固定；栈顶浮动；堆栈操作始终针对栈顶进行的。
- 进栈时，SP首先自动加1，将数据压入SP所指示的地址单元中；
- 出栈时，将SP所指示的地址单元中的数据弹出，然后SP自动减1。
- SP总是指向栈顶。
- 通常指定内部数据存储器地址07H—7FH中的一部分连续存储区作为堆栈。
- 系统复位后，SP初始化为07H，所以第一个压入堆栈的数据存放到08H单元。



## 2 输入/输出设备

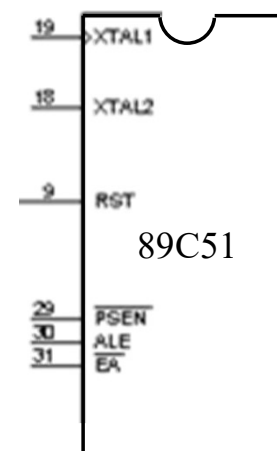
三类 { 电源及晶振引脚 (共4只)  
控制引脚 (共4只)  
端口引脚 (共32只)



## 2 输入/输出设备

### (2) 控制引脚

**RST/V<sub>PD</sub>** (9): 复位/ 备用电源引脚



**ALE/ $\overline{\text{PROG}}$**  (30): 地址锁存使能输出/ 编程脉冲输入

**$\overline{\text{PSEN}}$**  (29): 输出访问片外程序存储器读选通信号

**$\overline{\text{EA}}$ / V<sub>PP</sub>** (31): 外部ROM允许访问/ 编程电源输入

## 2 输入/输出设备

### (3) 端口引脚

**P0.0 ~ P0.7** (39 ~ 32脚) —— P0口

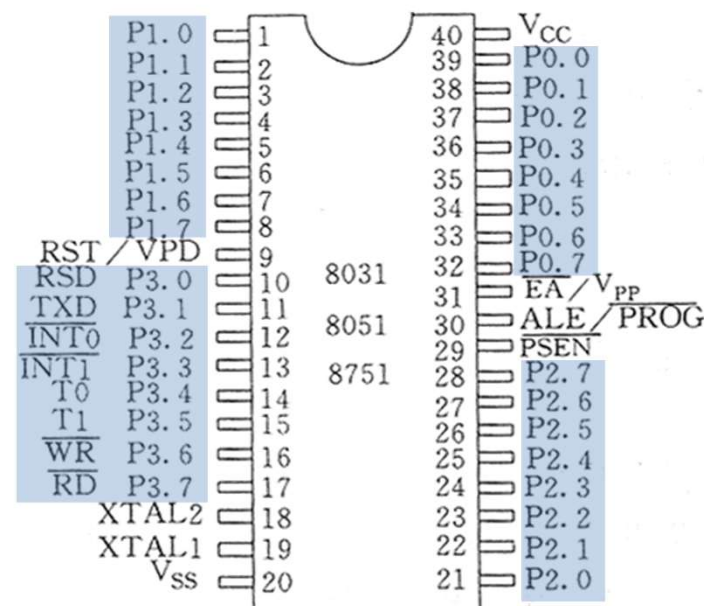
**P1.0 ~ P1.7** (1 ~ 8脚) —— P1口

**P2.0 ~ P2.7** (21 ~ 28脚) —— P2口

**P3.0 ~ P3.7** (10 ~ 17脚) —— P3口

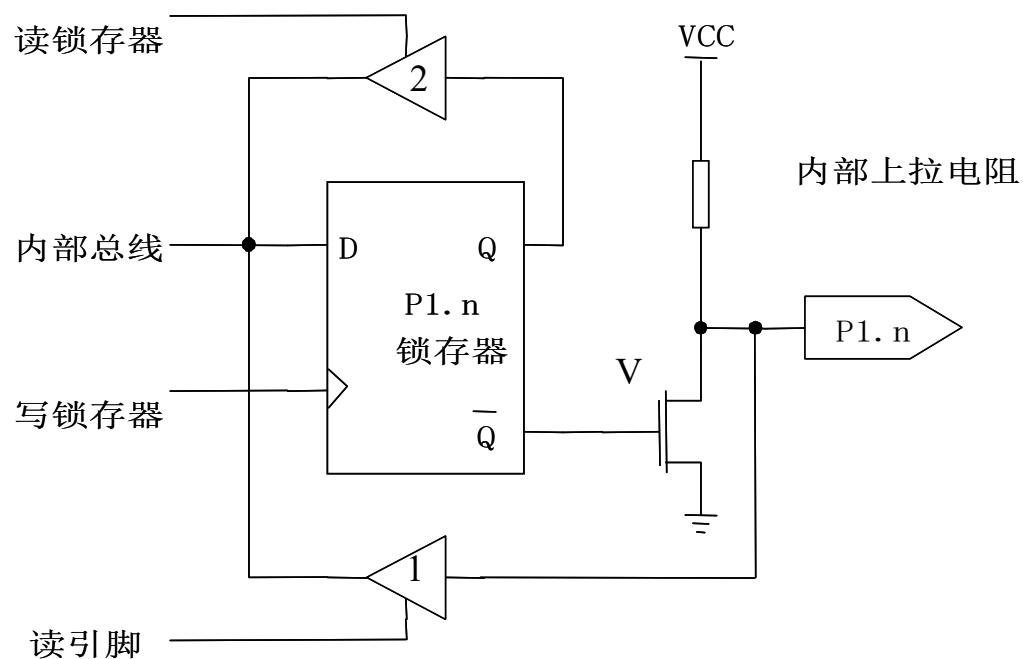
8只/组 × 4 组 = 32 只引脚

P0口 ~ P3口是单片机对外联络的重要通道



## 2 输入/输出设备

P1口包含**P1.0~P1.7**共8个相同结构的电路

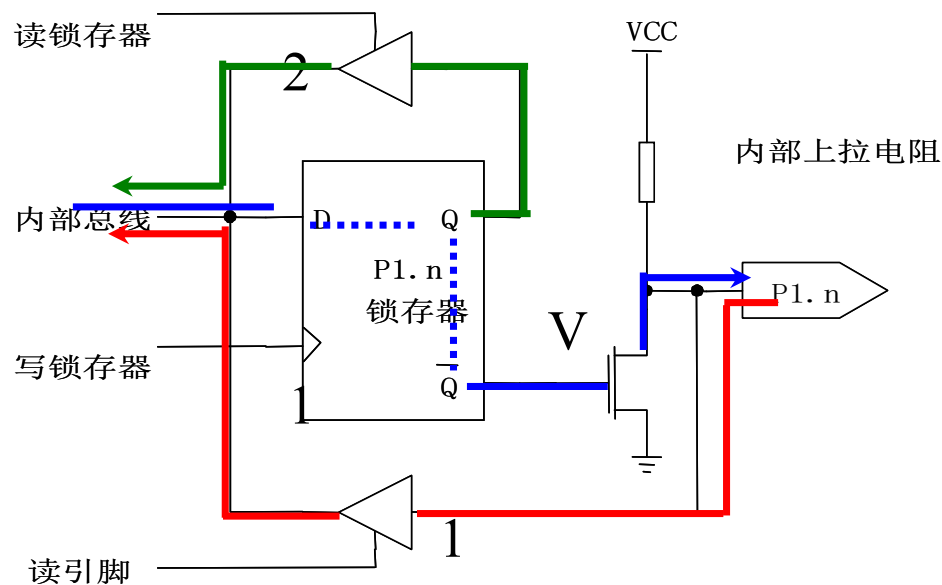


**P1.n** = 1个锁存器 + 1个场效应管驱动器V + 2个三态门缓冲器

P1.0~P1.7中的8个锁存器组成**P1 SFR (90H)**

## 2 输入/输出设备

P1.n的通用I/O口工作方式：输出、读引脚、读锁存器



输出时：  $D_{\text{端}}=1 \rightarrow /Q=0 \rightarrow V \text{截止} \rightarrow P1.n=1$

$D_{\text{端}}=0 \rightarrow /Q=1 \rightarrow V \text{导通} \rightarrow P1.n=0$

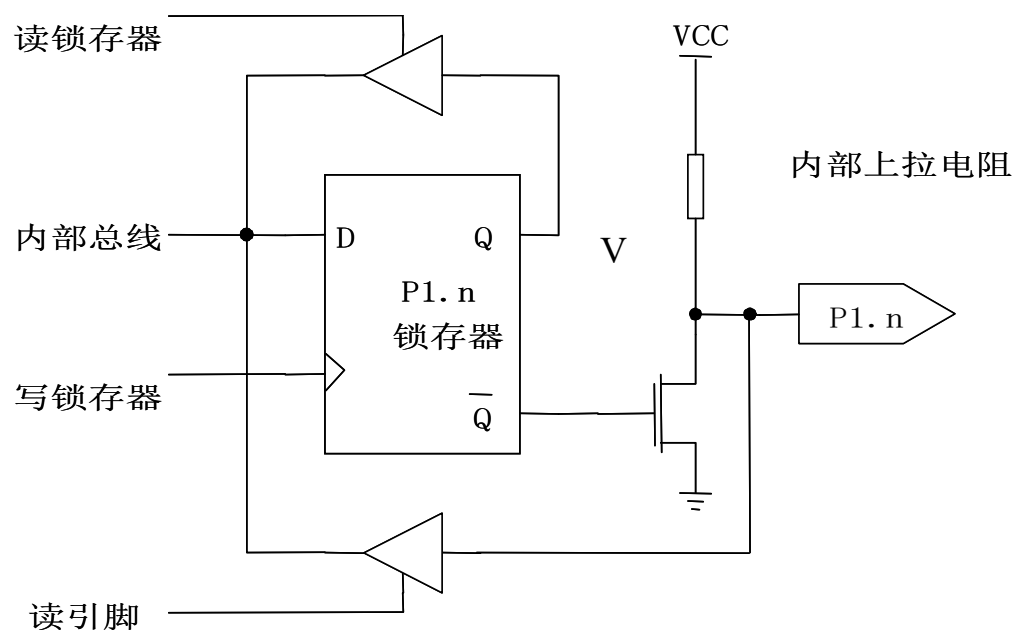
读引脚时：  $P1.n \rightarrow \text{读引脚三态门}1 \rightarrow \text{内部总线}$

读锁存器时：  $Q_{\text{端}} \rightarrow \text{读锁存器三态门}2 \rightarrow \text{内部总线}$

## 2 输入/输出设备

场效应管V的状态会影响P1.n的状态:

如V导通→P1.n电平≡0 (钳位) → 读引脚可能出错

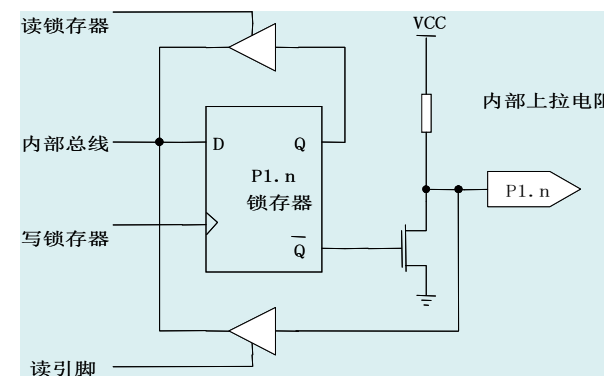
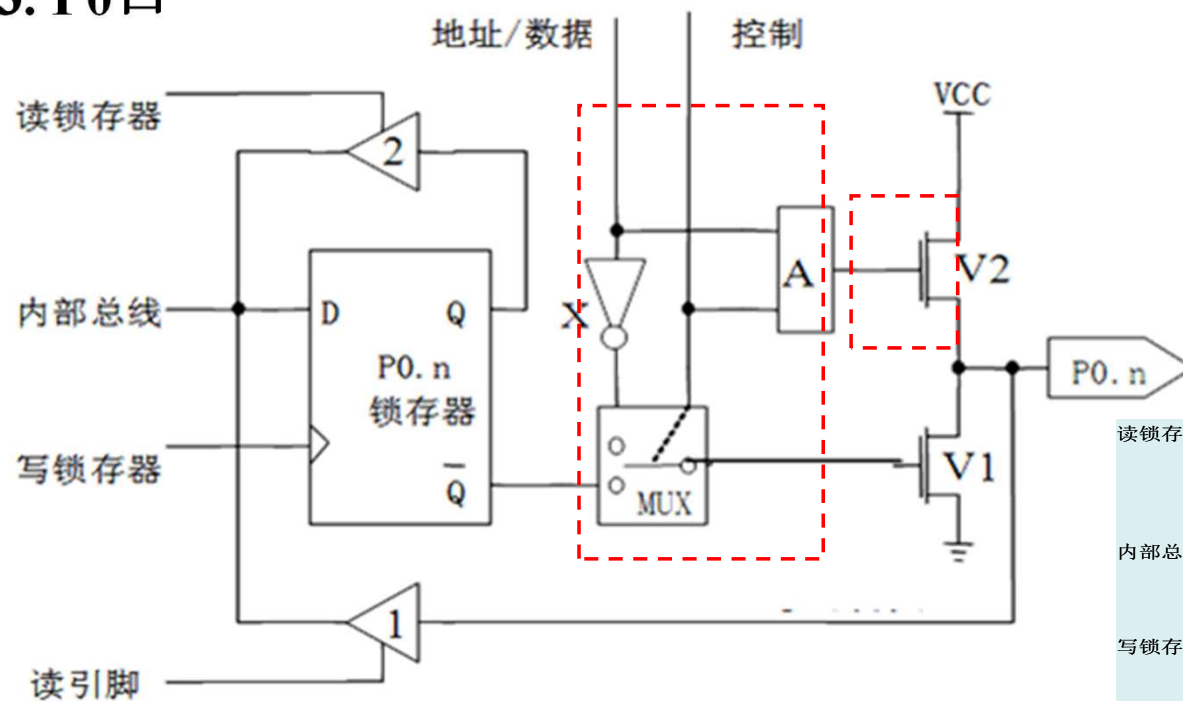


为正确读出P1.n引脚电平，需设法在读引脚前先使V截止  
令D=1→/Q=0→V截止→读P1.n→不会出错



## 2 输入/输出设备

### 3. P0口

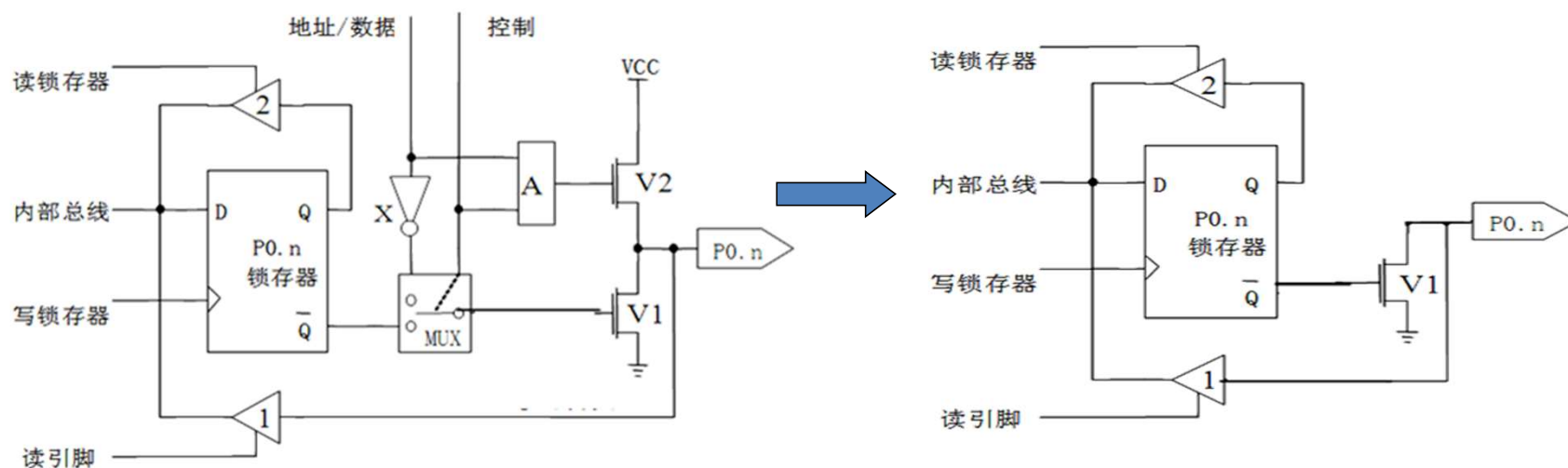


与P1.n 差别：输出控制电路、输出驱动电路→总线功能

P0.0~P0.7中的8个锁存器组成**P0 SFR (80H)**

## 2 输入/输出设备

### 漏极开路与上拉电阻的概念



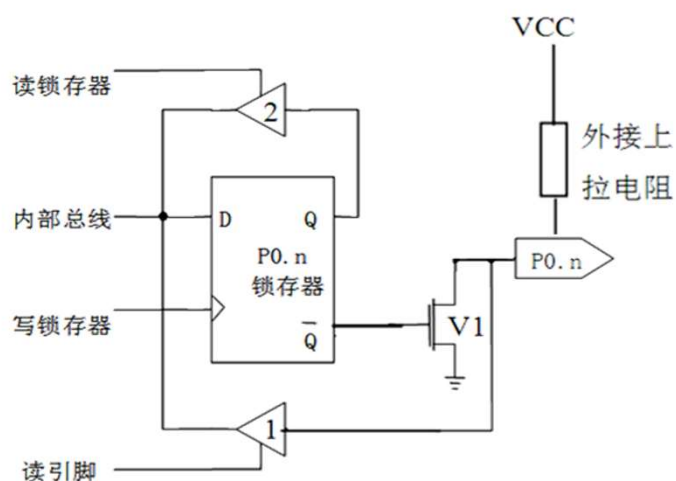
控制端=0→MUX下通→/Q与V1栅极直通

└→封锁与门A≡0→地址/数据端与A输出无关

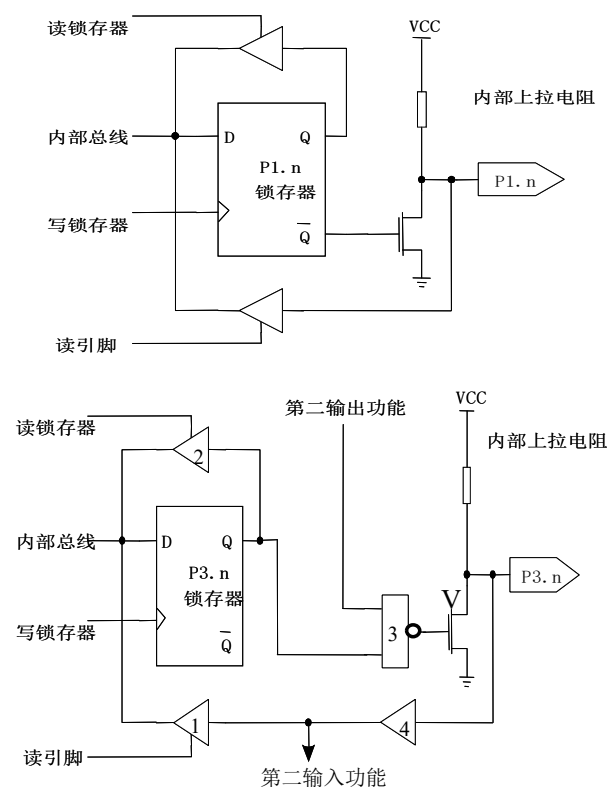
└→V2截止→V1漏极开路

## 2 输入/输出设备

为使漏极开路的V1有效，必须通过**外接上拉电阻**与电源连通，上拉电阻的阻值一般为 $10k\Omega$ 。



注意：P1、P2、P3口无需外接上拉电阻（已有内部上拉电阻）



## 2 输入/输出设备

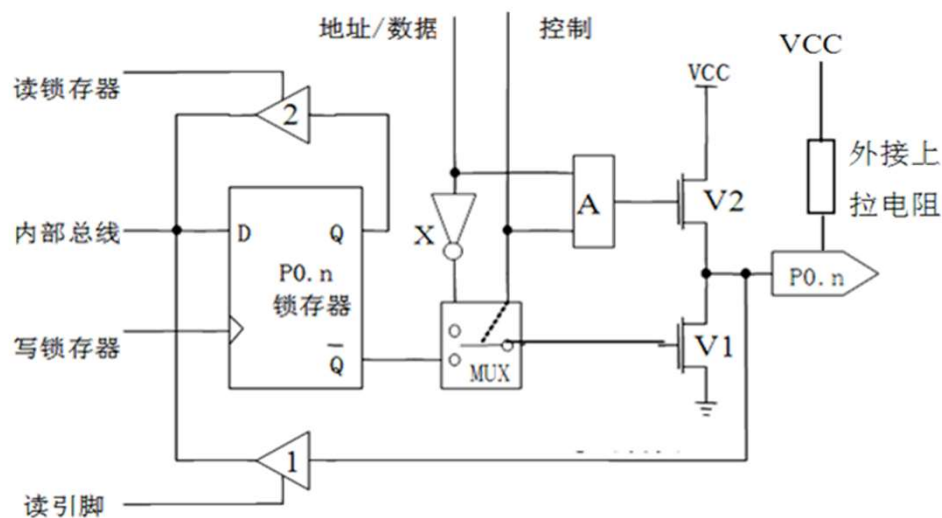
### P0.n的通用I/O口工作方式（控制端 = 0）

**输出时：** D端=1 $\rightarrow$ /Q=0 $\rightarrow$ V1截止 $\rightarrow$ P0.n=1

D端=0 $\rightarrow$ /Q/=1 $\rightarrow$ V1导通 $\rightarrow$ P0.n=0

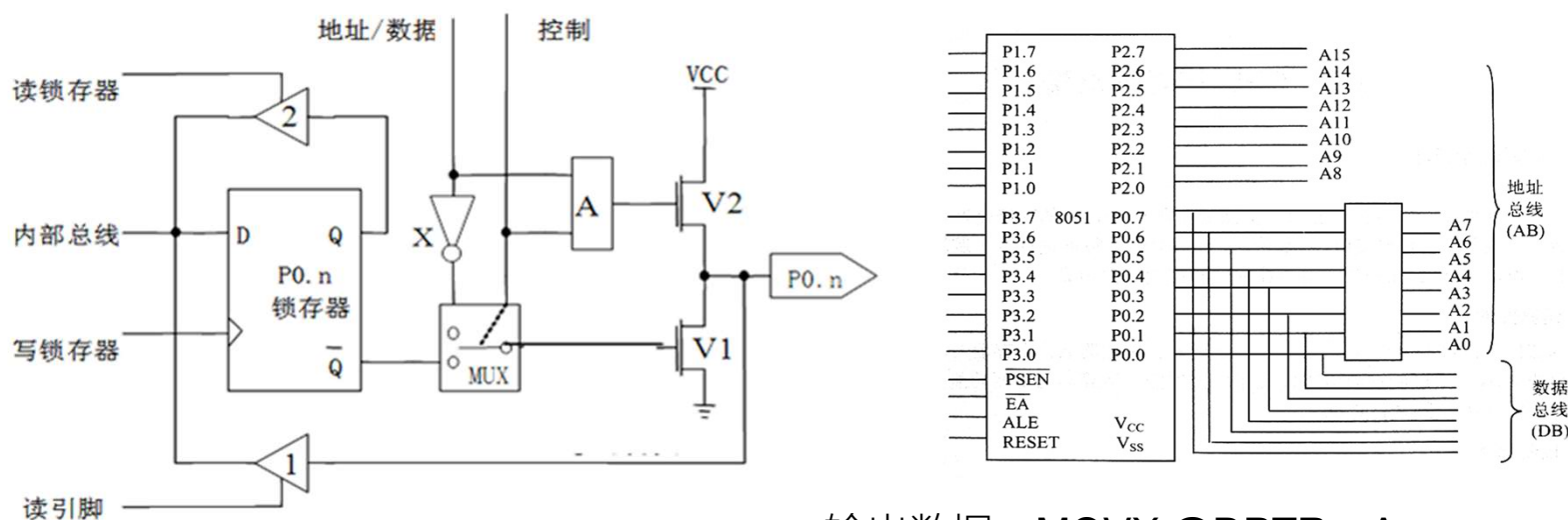
**读引脚时：** P0.n $\rightarrow$ 读引脚三态门1 $\rightarrow$ 内部总线（需要先写“1”）

**读锁存器：** Q端 $\rightarrow$ 读锁存器三态门2 $\rightarrow$ 内部总线



## 2 输入/输出设备

P0.n的**输出**地址/**输出**数据分时复用方式（**控制端=1**）



输出数据: `MOVX @DPTR, A`

➤ “**输出**地址/**输出**数据” 端可无条件输入/输出——（真正的）**双向口**

➤ “**输出**地址/**输出**数据” 方式下没有漏极开路问题，**无需外接上拉电阻**

## 2 输入/输出设备

P0.n的输入数据：读外部程序存储器或外部数据存储器

➤读外部程序存储器：在取指令期间，“控制”信号为“0”，V2管截止，多路开关也跟着转向锁存器反相输出端Q非；CPU自动将0FFH(11111111，即向D锁存器写入一个高电平‘1’)写入P0口锁存器，使V1管截止，在读引脚信号控制下，通过读引脚三态门电路将指令码读到内部总线。

➤读外部数据存储器：MOVX A, @DPTR(将外部RAM某一存储单元内容通过P0口数据总线输入到累加器A中)，则输入的数据仍通过读引脚三态缓冲器到内部总线

## 2 复位状态

| 寄存器   | 内容         | 寄存器  | 内容                |
|-------|------------|------|-------------------|
| PC    | 0000H      | TMOD | 00H               |
| A     | 00H        | TCON | 00H               |
| B     | 00H        | TH0  | 00H               |
| PSW   | 00H        | TL0  | 00H               |
| SP    | <b>07H</b> | TH1  | 00H               |
| DPTR  | 0000H      | TL1  | 00H               |
|       |            | SCON | 00H               |
| P0~P3 | <b>FFH</b> | SBUF | ××××××××B         |
| IP    | ×××0 0000B | PCON | 0××× 0000B(CHMOS) |
| IE    | 0××0 0000B |      | 0×××××××B(HMOS)   |

- 重点记忆不为0的两个复位状态；使用RST进行复位

## 2 单片机时序

机器周期： 1个机器周期=6个状态周期=12个节拍=12个振荡周期

指令长度与指令周期的联系：

指令长度取值为1字节，2字节，3字节；

指令周期取值为1机器周期，2机器周期，4机器周期；

单字节和双字节指令可能是单周期/双周期；

三字节指令都是双周期；

乘法/除法指令是唯二四周期指令；

所有的控制转移类指令都是双周期指令；

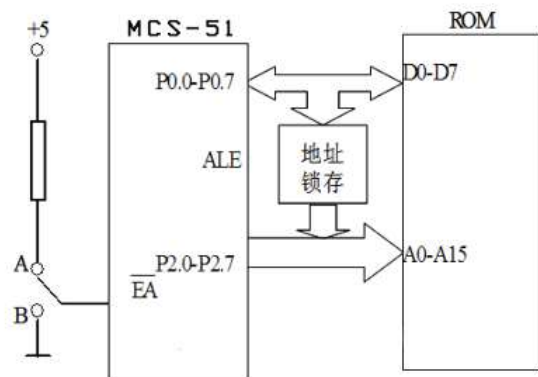


## 2 存储器

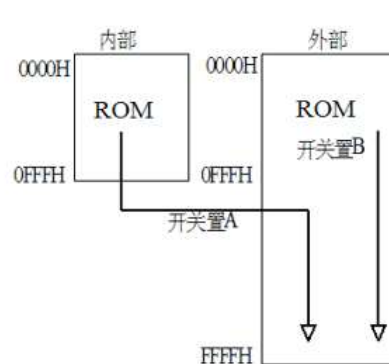
编址：51单片机采用哈佛结构，RAM、ROM分别编址。

一般可以认为，51单片机有以下逻辑存储空间：

片内外程序存储器空间，使用MOVC访问，片内片外统一编址，受到EA控制：



a) 同时使用片内和片外ROM



b) ROM地址分布

- 当**EA引脚接高电平**（开关接A点）时，4 KB以内的地址在片内ROM，大于4KB的地址在片外ROM中（图中折线），两者共同构成64KB空间；
- 当**EA引脚接低电平**（开关接B点）时，片内ROM被禁用，全部64KB地址都在片外ROM中（图中直线）。

2 存储器

片内数据存储器空间（256B，包括SFR），使用MOV访问。

片外数据存储器空间与片内独立编址，使用MOVX访问。

低128字节的区域：

- ①工作寄存器区（00H~1FH）
- ②可位寻址区（20H~2FH）
- ③用户RAM区（30H~7FH）

高128字节的区域：

每个存储单元都有一个**字节地址**，但只有其中**21个单元**可以使用，并有相应寄存器名称。

21个单元离散的分布在80H~FFH的范围内

|     |               |          |
|-----|---------------|----------|
| 7FH | 用户RAM区        |          |
|     | (堆栈、数据缓冲)     |          |
| 30H |               |          |
| 2FH | 位寻址区          |          |
|     | (位地址：00H~7FH) |          |
| 20H |               |          |
| 1FH | R7            | 第3组工作寄存器 |
| 18H | R0            |          |
| 17H | R7            | 第2组工作寄存器 |
| 10H | R0            |          |
| 0FH | R7            | 第1组工作寄存器 |
| 08H | R0            |          |
| 07H | R7            | 第0组工作寄存器 |
| 00H | R0            |          |

|     |      |        |
|-----|------|--------|
| FFH |      |        |
| F0H | B    |        |
| E0H | A    |        |
| D0H | PSW  |        |
| B8H | IP   |        |
| B0H | P3   |        |
| A8H | IE   |        |
| A0H | P2   |        |
| 99H | SBUF |        |
| 98H | SCON | 专用寄存器区 |
| 90H | P1   |        |
| 8DH | TH1  | SFR    |
| 8CH | TH0  |        |
| 8BH | TL1  |        |
| 8AH | TL0  |        |
| 89H | TMOD |        |
| 88H | TCON |        |
| 87H | PCON |        |
| 83H | DPH  |        |
| 82H | DPL  |        |
| 81H | SP   |        |
| 80H | P0   |        |

## 2 存储器

位寻址区每个单元有一个字节地址和八个位地址。





## 2 存储器

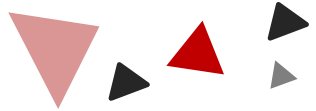
SFR中，字节地址可以被8整除的寄存器可以按位寻址。

常用的特殊功能寄存器**TMOD**，**PCON是不能位寻址**的，使用时只能整体赋值。

除此之外，SBUF、THx、TLx、DPH、DPL、SP也不能位寻址。

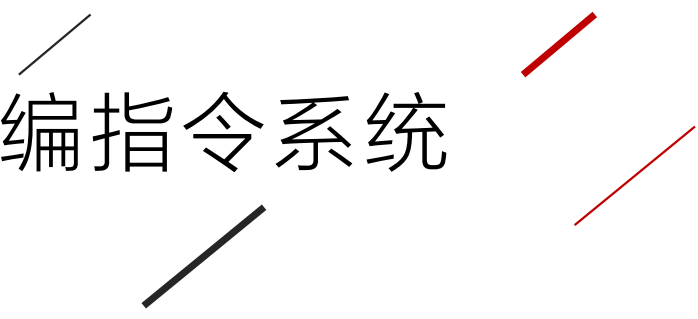
可以位寻址的特殊功能寄存器除了有字节地址之外还有位地址，在阅读32页表格时要注意区分。

# 3



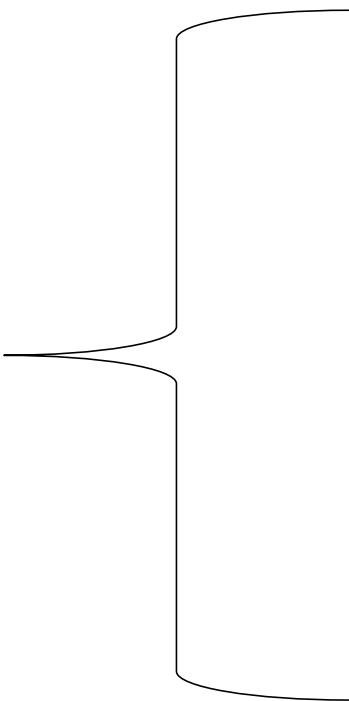
## Chapter

# 汇编指令系统





### 3 汇编指令系统



指令格式与简记符号

寻址方式

数据传送类指令

算术运算类指令

逻辑运算类指令

控制转移类指令

位操作类指令

汇编语言伪指令

### 3 指令格式与简记符号

#### 一、指令格式

一条指令最多包含如下四个区段：

|       |     |       |        |
|-------|-----|-------|--------|
| [标号：] | 操作码 | [操作数] | [； 注释] |
|-------|-----|-------|--------|

**START:**      **MOV**      **A, #12H**      ； 机器码**7412H**

标号：其值对应指令首字节存放地址，为转移指令提供目的地址

由用户定义的1~6个**英文字母与数字**组成，用：进行分隔。

操作码：标准注记字符，42个操作码，不分大小写。

操作数：目的操作数；源操作数

注释：以英文分号开始，无结束符号；不生成机器码。

### 3 指令格式与简记符号

#### 二、简记符号

指令手册中，每条指令的**操作数**是以**简记符号**表示的。

**Rn** : 当前寄存器组的**8**个通用寄存器**R0 ~ R7** (**n=0 ~ 7**) 。

**Ri** : 当前寄存器组中的**2**个寄存器**R0, R1**，用作间接寻址的寄存器 (**i=0,1**) 。

**direct** : 内部**RAM**的**8**位地址，指代**00H~FFH**。

**#data** : 包含在指令中的**8**位立即数。指代 **00H~FFH**。

**#data16**: 包含在指令中的**16**位立即数。指代 **0000H~FFFFH**。

**addr16** : **16**位目的地址， **64KB**存储器空间中的地址 。指代**0000H~FFFFH**。



### 3 指令格式与简记符号

#### 二、简记符号

指令手册中，每条指令的**操作数**是以**简记符号**表示的。

**addr11**：表示**11**位目的地址，用于**2KB**范围内的程序跳转。

**rel**：相对转移指令中的偏移量，为**8**位符号数。

**@**：寄存器间接寻址符号。

**/**：位操作数的前缀，表示对该位状态取反。

**(X)**：某寄存器或某单元中的内容。

**((X))**：以**X**单元的**内容为地址**的存储器单元内容，即**(X)**做地址，该地址单元的内容用**((x))**表示。

**←**：表示将箭尾一方的内容传送至箭头的一方。

### 3 寻址方式

寻址方式：寻找操作数地址或者指令地址的方式<相对操作数/地址而言>

#### 一、立即数寻址

- 操作数直接编码与指令中，立即数用#表示。如： **MOV A, #45H**

#### 二、直接寻址

- 在指令中直接给出了操作数的字节地址或者位地址。
- 使用范围：**SFR、片内RAM低128字节、位地址**。如： **MOV SP, #30H ; MOV C, 00H**

#### 三、寄存器寻址

- 操作数存于寄存器中
- 适用范围：**R0~R7、A、B（乘除法指令）、DPTR、C**
- B在乘除法以外指令属于**直接寻址**
- A也可以**直接寻址**，此时写作ACC。如： **PUSH ACC ;**
- 直接寻址的操作数所在字节地址占了指令一个字节，而寄存器寻址的寄存器是编在机器码中的，因此寄存器寻址往往指令长度更短。

### 3 寻址方式

寻址方式：寻找操作数地址或者指令地址的方式<相对操作数/地址而言>

如：MOV 30H, #30H 3字节；MOV A, #30H 2字节；MOV A, R7 1字节

#### 四、寄存器寻址

- 操作数的单元地址放在特定的寄存器中。
- 特定寄存器：R0、R1（RAM），DPTR（数据空间），SP（堆栈空间）

#### 五、变址寻址

- 操作数的有效地址是基址寄存器（PC、DPTR）与变址寄存器（A）内容之和
- **MOVC A, @A+DPTR**，在程序存储空间查表

#### 六、相对寻址

- 确定下一条执行指令的入口地址，指令中给出程序跳转偏移量rel（8bit符号数）
- 转移目的地址=PC当前值（注意PC指向要执行的下一条指令）+rel

#### 七、位寻址

- 在指令中直接给出操作数的位地址（适用片内位寻址区、SFR中支持位寻址的寄存器）
- 如MOV C, 00H；SETB PSW.5；CPL 20H.0；JBC RI, RECIV

### 3 寻址方式

### 寻址方式小结

| 寻址方式        | 使用符号                                                                   | 寻址空间                                           |
|-------------|------------------------------------------------------------------------|------------------------------------------------|
| 立即寻址        | #                                                                      | 程序存储器区                                         |
| 直接寻址        | direct                                                                 | 片内低 128BRAM 区 (00H-7FH);<br>SFR 区              |
| 寄存器寻址       | R0-R7<br>A<br>B (仅限于 乘、除法指令)<br>C (布尔寄存器)<br>DPTR<br>(仅限于 16bits 数据传送) | 工作寄存器 R0-R7<br>A, B, C, DPTR                   |
| 寄存器<br>间接寻址 | @ R0, @ R1                                                             | 片内 RAM (00H-7FH)<br>52 系列 MCU 高端 RAM (80H-FFH) |
|             | @ SP                                                                   | 堆栈                                             |
|             | @ R0, @ R1, @ DPTR                                                     | 片外 RAM 区                                       |
| 变址寻址        | A + DPTR; A + PC                                                       | 程序存储器区                                         |
| 相对寻址        | PC + rel                                                               | 程序存储器区                                         |
| 位寻址         |                                                                        | 片内 RAM 20H-2FH 单元;<br>SFR 区中的某些位               |

### 3 数据传送类指令

**MCS-51的指令系统共有111条指令**

**按指令字节长度分：**

单字节指令**49**条；  
双字节指令**46**条；  
三字节指令**16**条。

**按指令执行的时间分：**

- 单机器周期指令有**64**条；
- 双机器周期指令有**45**条；
- 四机器周期指令有**2**条。

**按功能可分为五类：**

- + **(1)数据传送类指令；**
- + **(2)算术运算类指令；**
- + **(3)逻辑运算类指令；**
- + **(4)程序控制类指令；**
- + **(5)位(布尔)操作类指令。**

### 3 数据传送类指令

| 类型     | 指令说明                | 助记符                         | 执行的操作                            |
|--------|---------------------|-----------------------------|----------------------------------|
| 片内数据传送 | 累加器A为目的操作数的传送       | <b>MOV A, Rn</b>            | $A \leftarrow (Rn)$              |
|        |                     | <b>MOV A, @Ri</b>           | $A \leftarrow ((Ri))$            |
|        |                     | <b>MOV A, #data</b>         | $A \leftarrow data$              |
|        |                     | <b>MOV A, direct</b>        | $A \leftarrow (direct)$          |
|        | 工作寄存器Rn为目的操作数的传送    | <b>MOV Rn, A</b>            | $Rn \leftarrow (A)$              |
|        |                     | <b>MOV Rn, #data</b>        | $Rn \leftarrow data$             |
|        |                     | <b>MOV Rn, direct</b>       | $Rn \leftarrow (direct)$         |
|        | 直接地址direct为目的操作数的传送 | <b>MOV direct, A</b>        | $(direct) \leftarrow (A)$        |
|        |                     | <b>MOV direct, Rn</b>       | $(direct) \leftarrow (Rn)$       |
|        |                     | <b>MOV direct, @Ri</b>      | $(direct) \leftarrow ((Ri))$     |
|        |                     | <b>MOV direct, #data</b>    | $(direct) \leftarrow data$       |
|        |                     | <b>MOV direct2, direct1</b> | $(direct2) \leftarrow (direct1)$ |
|        | 间接地址为目的操作数的传送       | <b>MOV @Ri, A</b>           | $((Ri)) \leftarrow (A)$          |
|        |                     | <b>MOV @Ri, direct</b>      | $((Ri)) \leftarrow (direct)$     |
|        |                     | <b>MOV @Ri, #data</b>       | $((Ri)) \leftarrow data$         |
|        | DPTR为目的操作数的传送       | <b>MOV DPTR, #data16</b>    | $DPTR \leftarrow data16$         |

### 3 数据传送类指令

| 类型            | 指令说明      | 助记符                    | 执行的操作                                          |
|---------------|-----------|------------------------|------------------------------------------------|
| 片外RAM<br>数据传送 | 写数据到片外RAM | <b>MOVX</b> @Ri, A     | $((Ri)) \leftarrow (A)$                        |
|               |           | <b>MOVX</b> @DPTR,A    | $((DPTR)) \leftarrow (A)$                      |
|               | 从片外RAM读数据 | <b>MOVX</b> A, @Ri     | $(A) \leftarrow ((Ri))$                        |
|               |           | <b>MOVX</b> A, @DPTR   | $(A) \leftarrow ((DPTR))$                      |
| ROM访问         |           | <b>MOVC</b> A, @A+PC   | $(A) \leftarrow ((A)+(PC))$                    |
|               |           | <b>MOVC</b> A, @A+DPTR | $(A) \leftarrow ((A)+(DPTR))$                  |
| 数据交换          |           | <b>XCH</b> A, Rn       | $A \leftrightarrow Rn$                         |
|               |           | <b>XCH</b> A, @Ri      | $A \leftrightarrow ((Ri))$                     |
|               |           | <b>XCH</b> A, direct   | $A \leftrightarrow (direct)$                   |
|               |           | <b>XCHD</b> A,@Ri      | $A0\sim3 \leftrightarrow ((Ri)_{0\sim3})$      |
|               |           | <b>SWAP</b> A          | $A0\sim3 \leftrightarrow A4\sim7$              |
| 堆栈操作          |           | <b>PUSH</b> direct     | $SP \leftarrow SP+1, ((SP)) \leftarrow direct$ |
|               |           | <b>POP</b> direct      | $direct \leftarrow ((SP)), SP \leftarrow SP-1$ |

### 3 数据传送类指令

注意事项:

——片内数据传送指令MOV:

- 立即数不能作为目的操作数, 即MOV #30H, A 非法;
- 工作寄存器不能同时作为目的操作数和源操作数, 即MOV R1, R2 和MOV R1, @R0非法;

——片外RAM数据传送MOVX:

- 必须通过累加器A, 即MOV @DPTR, 15H非法;
- 通过DPTR或Ri对片外间接寻址。
- 在MOVX A, @R0中, R0低八位由P0口送达, 默认使用P2端口传送存储单元高八位地址。因此, 使用时需要对P2端口置值。

——ROM访问:

- 使用MOVC访问;
- 采用变址寻址在ROM空间访问数据。MOVC A, @A+PC; MOVC A, @A+DPTR
- 基于DPTR查表指令范围64KB



### 3 数据传送类指令

例:字符**0-F**的**ASCII**码如下所示, 编写指令, 求出**4**的**ASCII**码, 结果存于**A**中。

**MOV A, #04H**

**MOV DPTR, # 0203H**

**MOVC A, @A+DPTR ;  $A \leftarrow ((A) + (DPTR))$**

执行前: **A=04H**

执行后:

**A = ((A)+(DPTR))**  
**= (04H+0203H)**  
**= (0207H)**  
**= 34H**

ASCII表  
首地址



| ROM地址 | 数值  |
|-------|-----|
| 0212H | 46  |
| 0211H | 45  |
| ...   | ... |
| 0209H | 36  |
| 0208H | 35  |
| 0207H | 34  |
| 0206H | 33  |
| 0205H | 32  |
| 0204H | 31  |
| 0203H | 30  |

### 3 数据传送类指令

注意事项：

——字节交换XCH：

➤ 把累加器A的内容与源操作数所指出的数据相互交换。

——半字节交换：

➤ 只有XCHD A,@Ri 一条指令。（必须以累加器A为目的操作数，源操作数间接寻址）

——高4位、低4位交换：

➤ SWAP A 操作数必须是累加器A。

——堆栈操作指令PUSH、POP：

| 助记符格式       | 操作                                                 |
|-------------|----------------------------------------------------|
| PUSH direct | $SP \leftarrow SP + 1, ((SP)) \leftarrow (direct)$ |
| POP direct  | $(direct) \leftarrow ((SP)), SP \leftarrow SP - 1$ |

➤ 采用直接寻址。因此累加器A入栈是PUSH ACC。

### 3 数据传送类指令

## 数据传送类指令对PSW的影响

一般情况下不影响Cy，OV，AC，P标志，但存在以下特例。

特例1：目的操作数为A时，影响P标志

如：MOV A, #40H

特例2：目的操作数为PSW时，改变PSW中相应标志

如：MOV PSW, #00001000B

### 3 算术运算类指令

| 类型               | 指令说明      | 助记符                   | 执行的操作                              |
|------------------|-----------|-----------------------|------------------------------------|
| 加 法 指 令<br>加 类 令 | 加法指令      | <b>ADD A, Rn</b>      | $A \leftarrow A + (Rn)$            |
|                  |           | <b>ADD A, @Ri</b>     | $A \leftarrow A + ((Ri))$          |
|                  |           | <b>ADD A, #data</b>   | $A \leftarrow A + data$            |
|                  |           | <b>ADD A, direct</b>  | $A \leftarrow A + (direct)$        |
|                  | 带进位加法指令   | <b>ADDC A, Rn</b>     | $A \leftarrow A + (Rn) + Cy$       |
|                  |           | <b>ADDC A, @Ri</b>    | $A \leftarrow A + ((Ri)) + Cy$     |
|                  |           | <b>ADDC A, #data</b>  | $A \leftarrow A + data + Cy$       |
|                  |           | <b>ADDC A, direct</b> | $A \leftarrow A + (direct) + Cy$   |
|                  | 加1指令      | <b>INC A</b>          | $A \leftarrow A + 1$               |
|                  |           | <b>INC Rn</b>         | $Rn \leftarrow Rn + 1$             |
|                  |           | <b>INC direct</b>     | $(direct) \leftarrow (direct) + 1$ |
|                  |           | <b>INC @Ri</b>        | $((Ri)) \leftarrow ((Ri)) + 1$     |
|                  |           | <b>INC DPTR</b>       | $DPTR \leftarrow DPTR + 1$         |
|                  | 二-十进制调整指令 | <b>DA A</b>           | 修正BCD码结果                           |

### 3 算术运算类指令

| 类型    | 指令说明     | 助记符            | 操作                                                            |
|-------|----------|----------------|---------------------------------------------------------------|
| 减法指令  | 带借位的减法指令 | SUBB A, Rn     | $A \leftarrow A - (Rn) - C_Y$                                 |
|       |          | SUBB A, direct | $A \leftarrow A - (\text{direct}) - C_Y$                      |
|       |          | SUBB A, @Ri    | $A \leftarrow A - ((Ri)) - C_Y$                               |
|       |          | SUBB A, #data  | $A \leftarrow A - \text{data} - C_Y$                          |
|       | 减一指令     | DEC A          | $A \leftarrow A - 1$                                          |
|       |          | DEC Rn         | $Rn \leftarrow Rn - 1$                                        |
|       |          | DEC direct     | $(\text{direct}) \leftarrow (\text{direct}) - 1$              |
|       |          | DEC @Ri        | $((Ri)) \leftarrow ((Ri)) - 1$                                |
| 乘法类指令 |          | MUL AB         | $A * B \rightarrow BA$                                        |
| 除法类指令 |          | DIV AB         | $A \div B \text{ 的商} \rightarrow A, \text{ 余数} \rightarrow B$ |

### 3 算术运算类指令

注意事项：

——加法指令、带进位加法指令：

➤ A必须是目的操作数。

——加一/减一指令：

➤ INC/DEC direct针对的是IO端口，执行的是读（锁存器）-改（加/减1）-写（锁存器）操作。

——二进制调整指令：

➤ 只能跟在ADD与ADDC后（INC不行），不适用于减法。

——乘法指令：

➤ 两个8位无符号数，结果高八位存于B，低八位存于A；

➤ 乘积大于255，OV=1，否则清零；Cy总为0；

——除法指令：

➤ 结果整数存放于A，小数存放于B；

➤ 如果除数为0，结果不定，OV=1；Cy总为0

### 3 算术运算类指令

例：编写将累加器A中十六进制数转换成3位BCD码程序，结果的百位数存于R7。十位数和个位数存于R6。(胡-例3.3.7)

解：应用除法指令，将待转换的数除以100，得百位数，再将余数除以10，得十位数的余数，即为个位数。编写程序如下：

```
MOV    B, #100
DIV     AB           ; A中商为百位数
MOV     R7, A        ; 百位数送R7

MOV     A, #10
XCH     A, B         ; B中余数与A中除数10互换
DIV     AB           ; A中得十位数, B中得个位数

SWAP    A
ADD     A, B         ; 组合成2位BCD码
MOV     R6, A        ; 十位、个位数送R6
SJMP    $
```

## 算术运算指令小结

### **ADD、ADDC、SUBB:**

适用于符号数和无符号数的运算。

每次执行后均会刷新Cy, OV, AC, P标志。

实现多字节加减法:

使用ADDC 和SUBB指令。

实现多字节BCD码的加法:

使用 ADDC 和 DA。

### **INC、DEC指令:**

不改变Cy, AC的状态, 因此不要与DA指令配合使用。



## 3.3.3 逻辑运算类指令

- 逻辑运算:

逻辑与**ANL**、逻辑或**ORL**、逻辑异或**XRL**

取反**CPL**、清零**CLR**

循环移位**RR**、**RRC**、**RL**、**RLC**。

### 3.3.3 逻辑运算类指令

#### 一、逻辑与指令

| 助记符格式             | 操作                                                              |
|-------------------|-----------------------------------------------------------------|
| ANL A, Rn         | $A \leftarrow A \wedge Rn$                                      |
| ANL A, direct     | $A \leftarrow A \wedge (\text{direct})$                         |
| ANL A, @Ri        | $A \leftarrow A \wedge ((Ri))$                                  |
| ANL A, #data      | $A \leftarrow A \wedge \text{data}$                             |
| ANL direct, A     | $(\text{direct}) \leftarrow (\text{direct}) \wedge A$           |
| ANL direct, #data | $(\text{direct}) \leftarrow (\text{direct}) \wedge \text{data}$ |

### 3.3.3 逻辑运算类指令

例如：已知 **A = 8DH**，**R0 = 7EH**，执行指令：**ANL A, R0**

$$\begin{array}{r} 1000\ 1101\ (8DH) \\ \wedge) \ 0111\ 1110\ (7EH) \\ \hline 0000\ 1100\ (0CH) \end{array}$$

结果：**A = 0CH**。

功能：

用**ANL**指令屏蔽（清零）某些位，即将需屏蔽的位和**0**相与。

## 3.3.3 逻辑运算类指令

### 二、逻辑或指令

| 助记符格式             | 操作                                                            |
|-------------------|---------------------------------------------------------------|
| ORL A, Rn         | $A \leftarrow A \vee Rn$                                      |
| ORL A, direct     | $A \leftarrow A \vee (\text{direct})$                         |
| ORL A, @Ri        | $A \leftarrow A \vee ((Ri))$                                  |
| ORL A, #data      | $A \leftarrow A \vee \text{data}$                             |
| ORL direct, A     | $(\text{direct}) \leftarrow (\text{direct}) \vee A$           |
| ORL direct, #data | $(\text{direct}) \leftarrow (\text{direct}) \vee \text{data}$ |

### 3.3.3 逻辑运算类指令

例如: 已知A = C0H, R0 = 55H, 执行指令 ORL A, R0。

$$\begin{array}{r} 1100\ 0000\ (C0H) \\ \vee) \ 0101\ 0101\ (55H) \\ \hline 1101\ 0101\ (D5H) \end{array}$$

- 逻辑或指令可以实现置位的功能。
- 同时逻辑或指令也可用于组合信息, 如字节拼装。

## 3.3.3 逻辑运算类指令

### 三、逻辑异或指令

| 助记符格式             | 操作                                                              |
|-------------------|-----------------------------------------------------------------|
| XRL A, Rn         | $A \leftarrow A \oplus Rn$                                      |
| XRL A, direct     | $A \leftarrow A \oplus (\text{direct})$                         |
| XRL A, @Ri        | $A \leftarrow A \oplus ((Ri))$                                  |
| XRL A, #data      | $A \leftarrow A \oplus \text{data}$                             |
| XRL direct, A     | $(\text{direct}) \leftarrow (\text{direct}) \oplus A$           |
| XRL direct, #data | $(\text{direct}) \leftarrow (\text{direct}) \oplus \text{data}$ |

### 3.3.3 逻辑运算类指令

例：已知 **A = A5H**，要求对其高4位取反。  
执行指令 **XRL A, #11110000B**

$$\begin{array}{r} 1010\ 0101\ (\text{A5H}) \\ )\ 1111\ 0000\ (\text{F0H}) \\ \hline 0101\ 0101\ (\text{55H}) \end{array}$$

结果：**A = 55H**。

**XRL**操作可用于对某些位取反。  
**XRL**等同于不进位加法。

### 3.3.3 逻辑运算类指令

#### 四、累加器A清零指令

**CLR A** ;  $A \leftarrow 0$

将累加器A的内容清0。

#### 五、累加器A取反指令

**CPL A** ;  $\overline{A} \leftarrow A$

将累加器A的内容逐位取反。

字节清零、取反操作仅限于A。



### 3.3.3 逻辑运算类指令

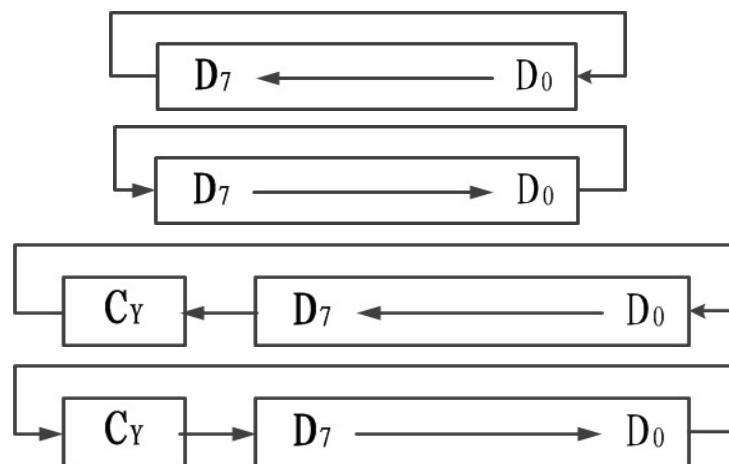
- 六、累加器**A**循环移位指令

– RL     A

– RR     A

– RLC    A

– RRC    A



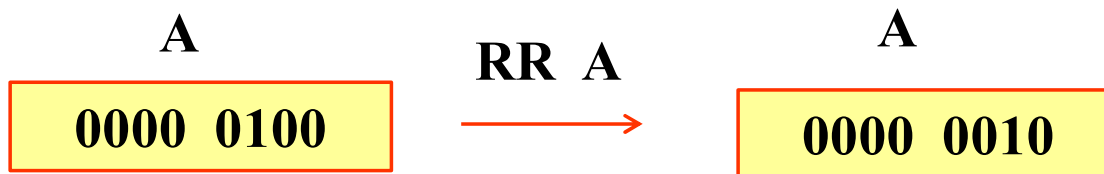
**RL / RR:** **A**中内容向左 或向右自循环一位。

**RLC / RRC:** 将 **A** 的内容连同 **CY** 循环左移或右移一位。

限制：循环指令仅限于**A**

### 3.3.3 逻辑运算类指令

如何利用循环移位实现乘2，或除2运算？



### 3.3.3 逻辑运算类指令

问题拓展：多字节数右移一位如何实现？

| B         | A         |
|-----------|-----------|
| 1000 0001 | 0000 0010 |

| B         | A         |
|-----------|-----------|
| 0100 0000 | 1000 0001 |

无符号数右移1位

| B         | A         |
|-----------|-----------|
| 1100 0000 | 1000 0001 |

符号数右移1位时最高位要补1，保证符号位性质不变！

### 3.3.3 逻辑运算类指令

#### 逻辑运算指令的小结

限制:

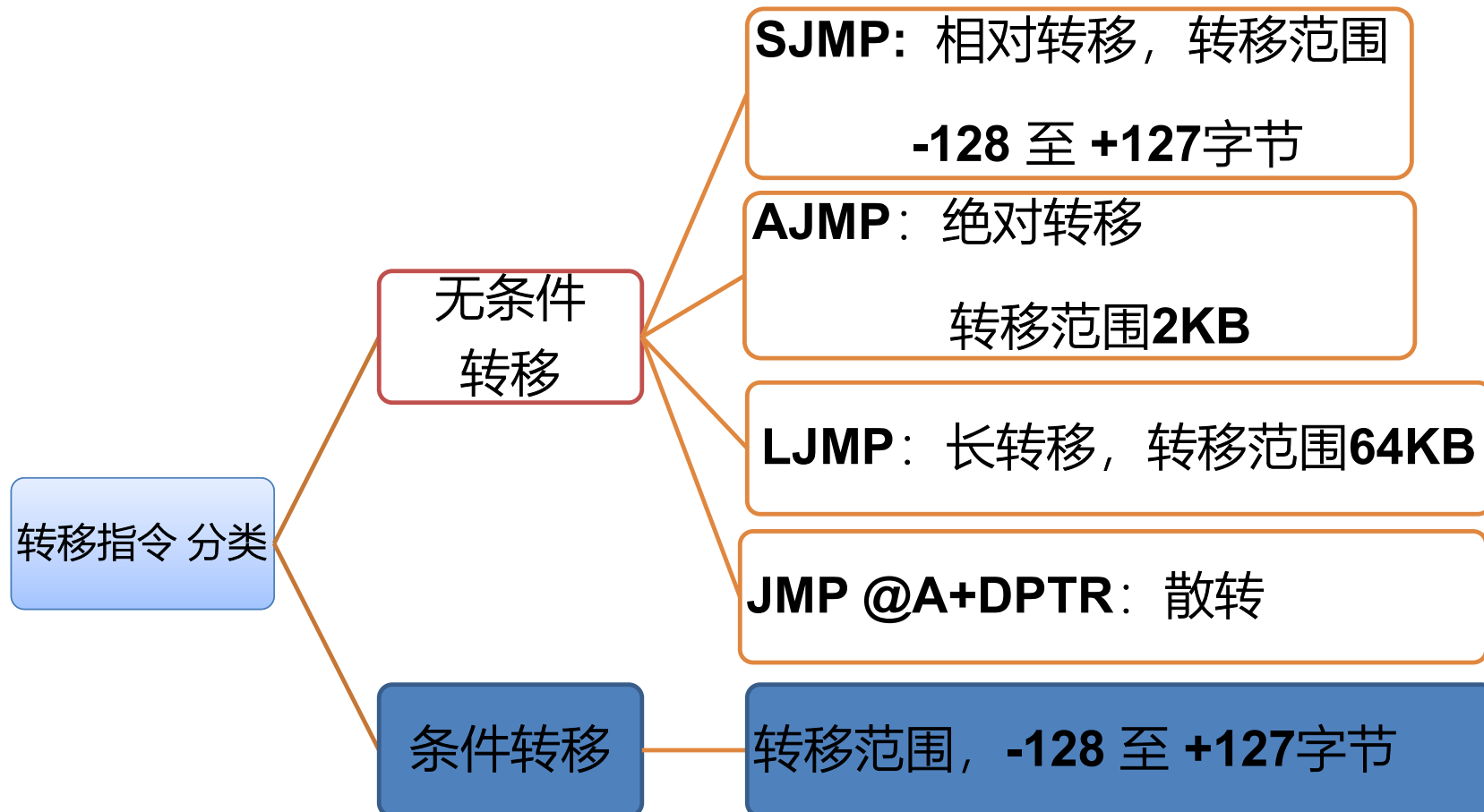
1. **ANL, ORL, XRL**指令  
第一操作数只能是**A**, 或**direct**;
2. **RL/RR/RLC/RRC** 仅仅适用于**A**。
3. 针对**字节**操作的**CLR、CPL**指令仅仅适用于**A**。

### 3.3.3 逻辑运算类指令

#### 逻辑指令的典型应用

1. 对字节中的特定位进行清零、置1，或取反。
2. 字节拼装（非压缩**BCD**码 → 压缩**BCD**码）
3. 字节移位（单字节、多字节；左移、右移；无符号数、符号数）
4. 求补运算（单字节、多字节）

### 3.3.4 控制转移类指令



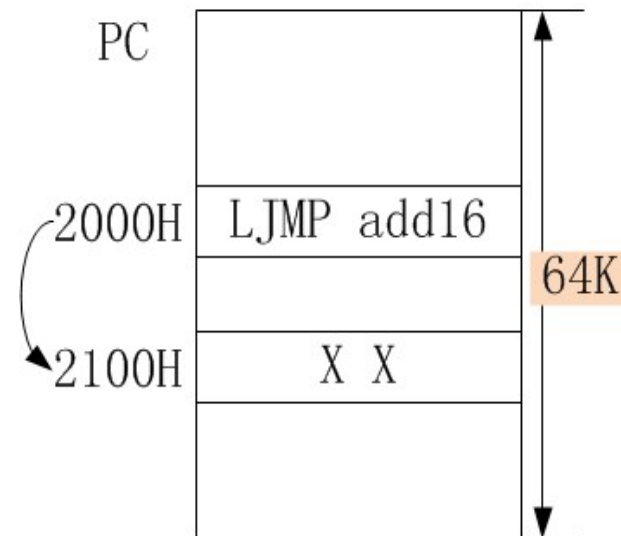
### 3.3.4 控制转移类指令

#### 1、无条件转移指令 (4条)

- 1) 长转移指令 (3字节)

**LJMP**            **addr16**        ; PC←addr16

举例    **2000H: LJMP    2100H        ; PC=2100H**

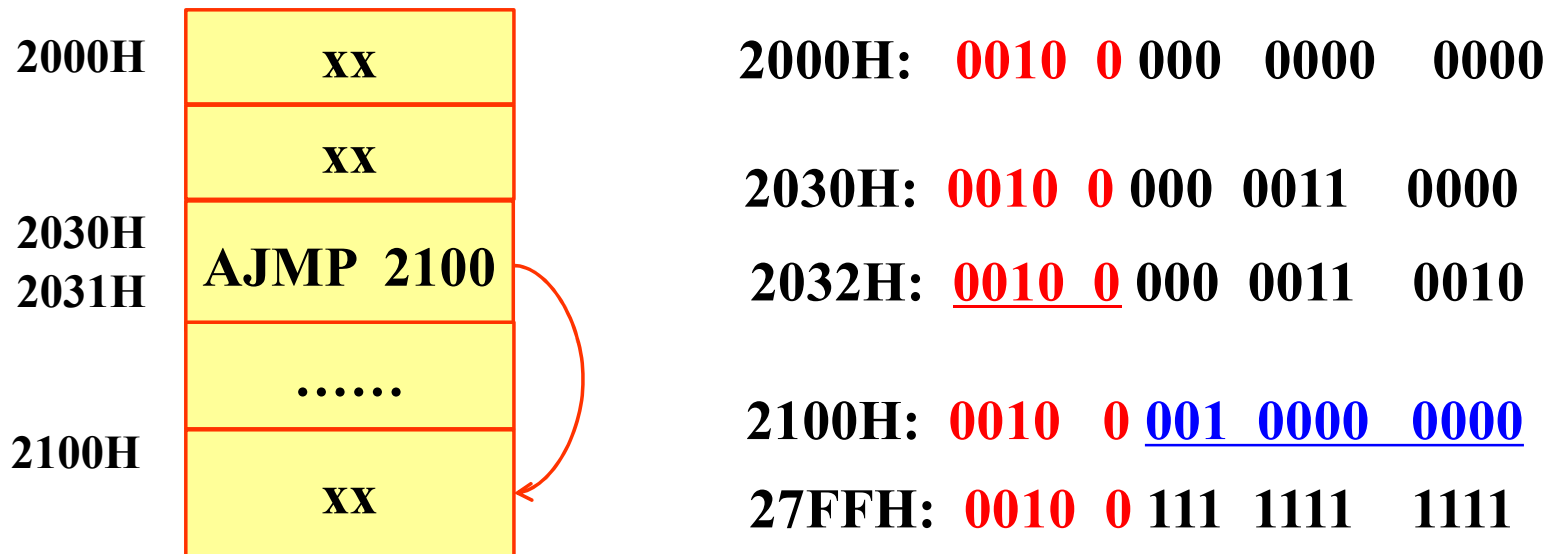


### 3.3.4 控制转移类指令

#### 2) 绝对转移指令 (2字节)

**AJMP addr11 ; PC←PC+2, PC.10~PC.0←addr11**

举例      **2030H: AJMP      2100H**





### 3.3.4 控制转移类指令

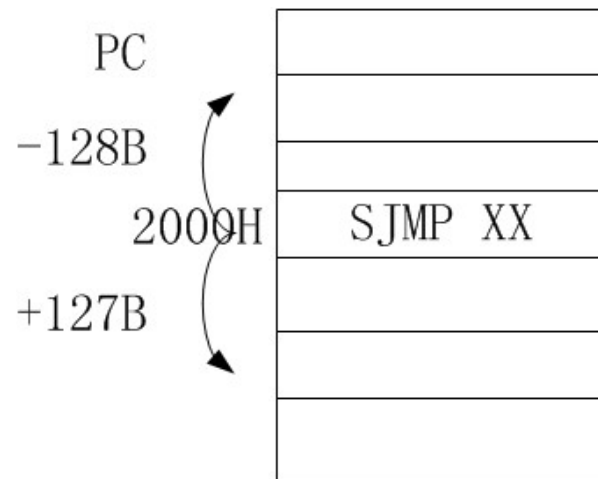
限制:

- **AJMP** 指令的转移范围是**2KB**。
- 要求转移目的地址与 **AJMP** 指令的下一条指令位于同一个**2KB**区域内, 即它们的高**5**位地址相同。

### 3.3.4 控制转移类指令

#### 3) 相对转移指令 (2字节)

**SJMP rel ;  $PC \leftarrow PC + 2 + rel$**



### 3.3.4 控制转移类指令

- **MCS-51**单片机的指令系统中没有停机指令，通常使用**SJMP**构成踏步指令使程序“原地踏步”，反复执行**SJMP** 指令。

例如： **HERE: SJMP HERE**  
或 **SJMP \$**

- 其中 **\$**: 符号地址，代表该指令的首地址
- 若有中断发生，或者**CPU**复位，就可以脱离该状态。

### 3.3.4 控制转移类指令

#### 4) 间接寻址的无条件转移指令

**JMP    @A+DPTR ; PC←(A)+(DPTR)**

常用于多分支选择转移,

由**DPTR**内容决定多分支程序转移表的首地址,

由**A**的内容选择其中的某一个分支转移程序(或指令),

从而使用一条间接转移指令就可以代替多条转移指令,

具有散转功能, 因而又称散转指令。

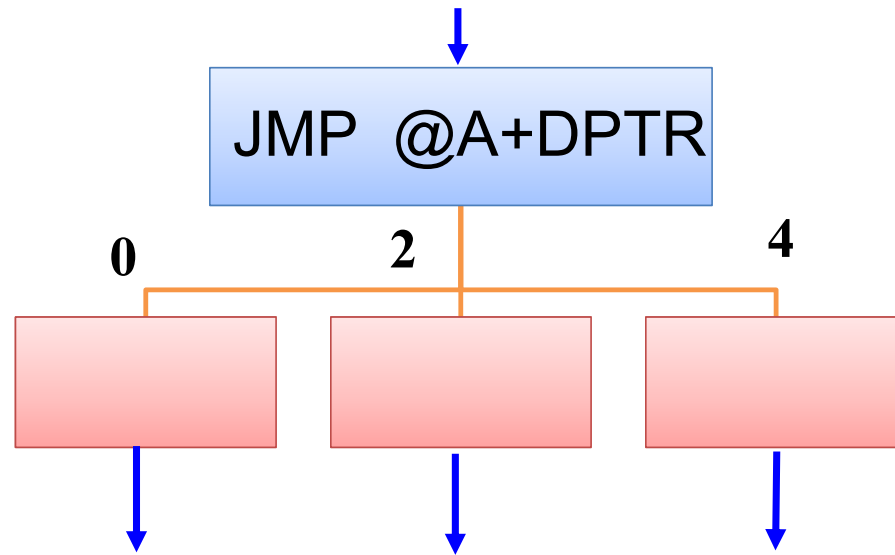
### 3.3.4 控制转移类指令

**JMP @A+DPTR** 的特点:

- 变址寻址转移指令与前述两条转移指令的主要区别是：
  - 前述两条指令的转移目标地址是在汇编或编程时已确定的;
  - 该指令的转移目标地址是在程序运行时动态决定的, 它的目标地址是以**DPTR**的内容为起点的**256**个字节地址空间范围内的指定地址。

### 3.3.4 控制转移类指令

- 例：设**A**为0~4之间的偶数，执行程序，根据**A**的内容实现散转。



### 3.3.4 控制转移类指令

| LOC             | OBJ     | 源程序                |
|-----------------|---------|--------------------|
| 2000H: XX XX XX |         | MOV DPTR, #JPTBL   |
| 2003H: XX       |         | <b>JMP @A+DPTR</b> |
| 2004H: XX XX    | JPTBL:  | AJMP LABEL0        |
| 2006H: XX XX    |         | AJMP LABEL1        |
| 2008H: XX XX    |         | AJMP LABEL2        |
|                 | LABEL0: | INC A              |
|                 |         | SJMP \$            |
|                 |         | .....              |



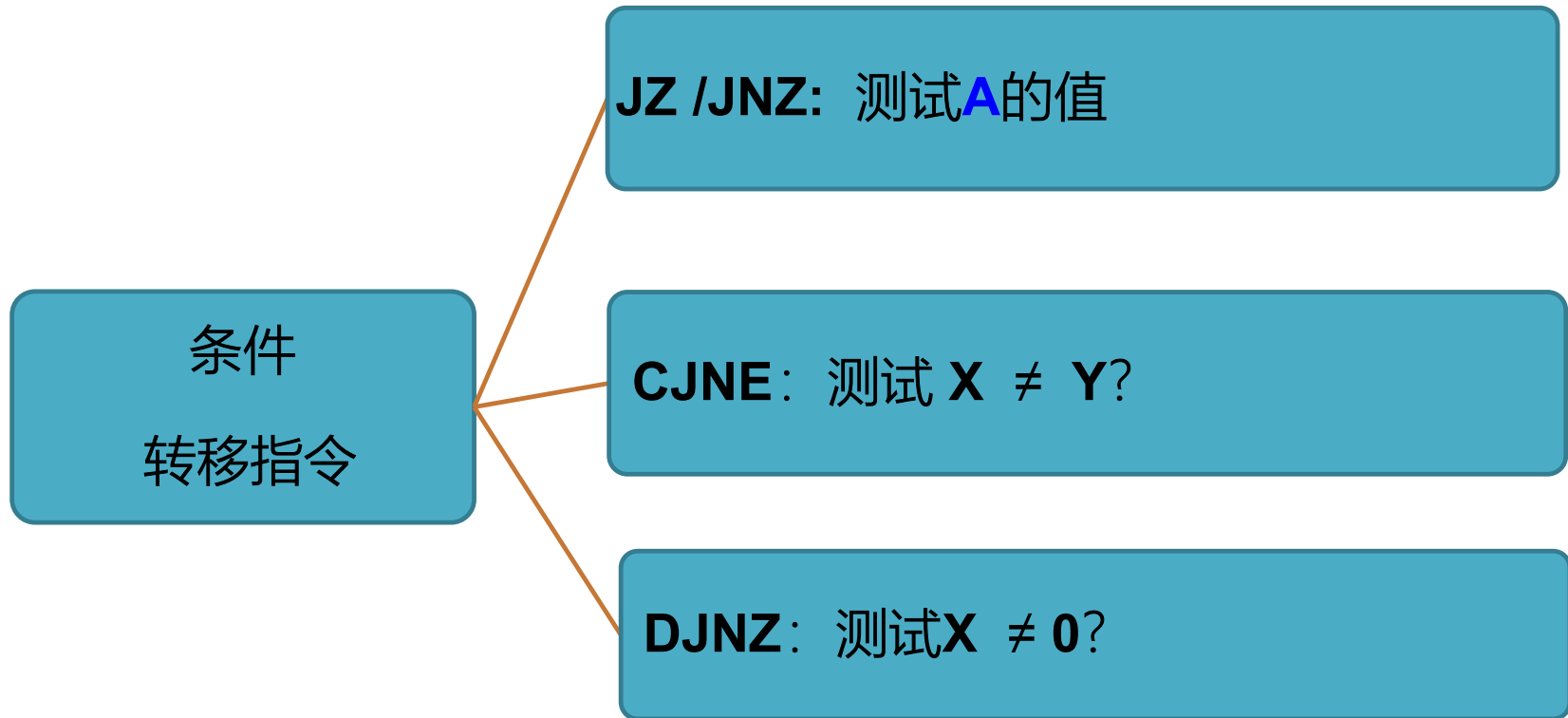
标号JPTBL，符号地址，其值 = 2004H，故 DPTR = 2004H  
A=2时，PC = (DPTR) + (A) = 2004H + 02H = 2006H;

### 3.3.4 控制转移类指令

- 2. 条件转移指令(10条)
  - 根据给定的条件进行检测，  
若条件得到满足，则程序转向指定的目标地址去执行；  
否则，不转移，继续往下顺序执行程序。
  - 条件转移指令都是相对转移指令。  
其转移的范围是以转移指令的下一条指令的第一个字节地址为起始地址的 -128 ~ +127 个字节内。



### 3.3.4 控制转移类指令

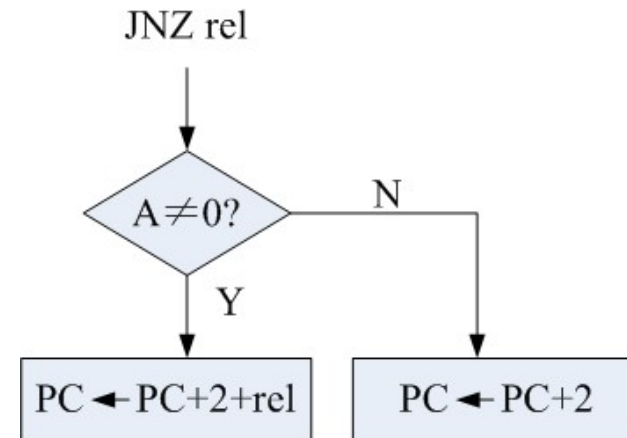
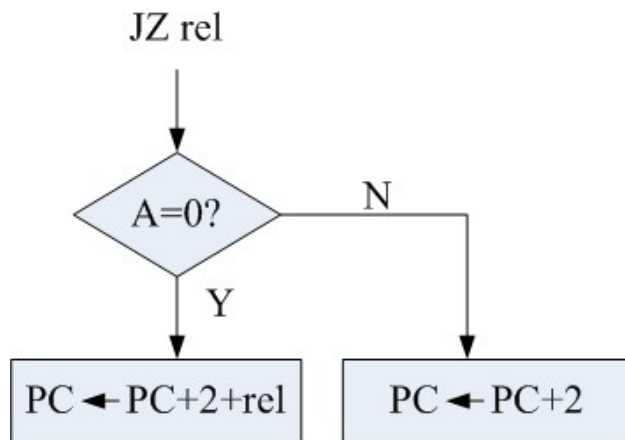


### 3.3.4 控制转移类指令

- 1) 累加器A判零转移指令

JZ rel;  $PC=PC+2$ , if  $A=0$ , then  $PC=PC+rel$

JNZ rel;



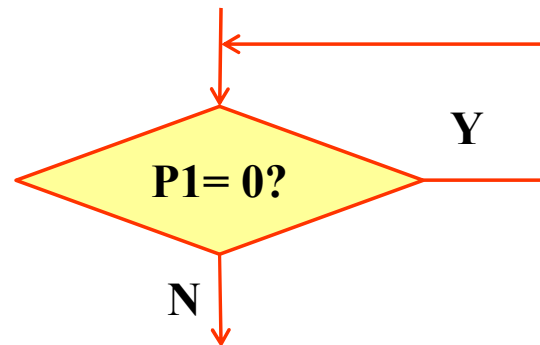
### 3.3.4 控制转移类指令

- 例：测试A的内容是否为0

AGAIN:     MOV    A, P1   ; 读端口引脚

**JZ    AGAIN**

          MOV   20H, A   ; 非0则保存数据



### 3.3.4 控制转移类指令

- 2) 比较转移指令 (判断  $X = Y?$  )

CJNE      A, direct, rel

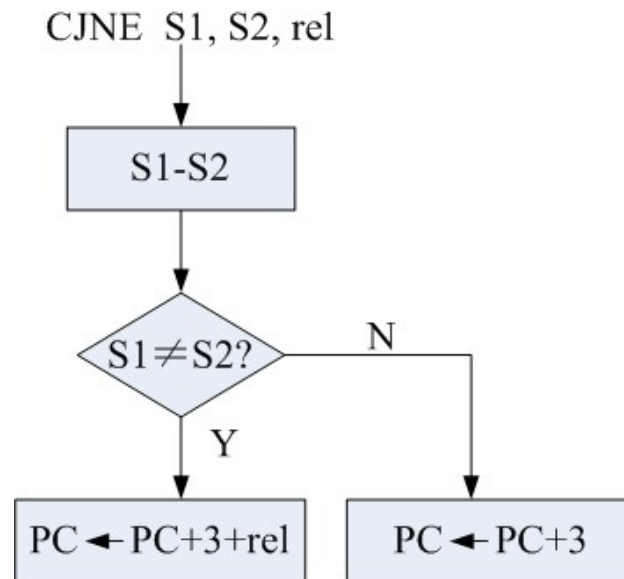
CJNE      A, #data, rel

CJNE       $R_n$ , #data, rel

CJNE       $@R_i$ , #data, rel

### 3.3.4 控制转移类指令

- **CJNE**是三字节指令



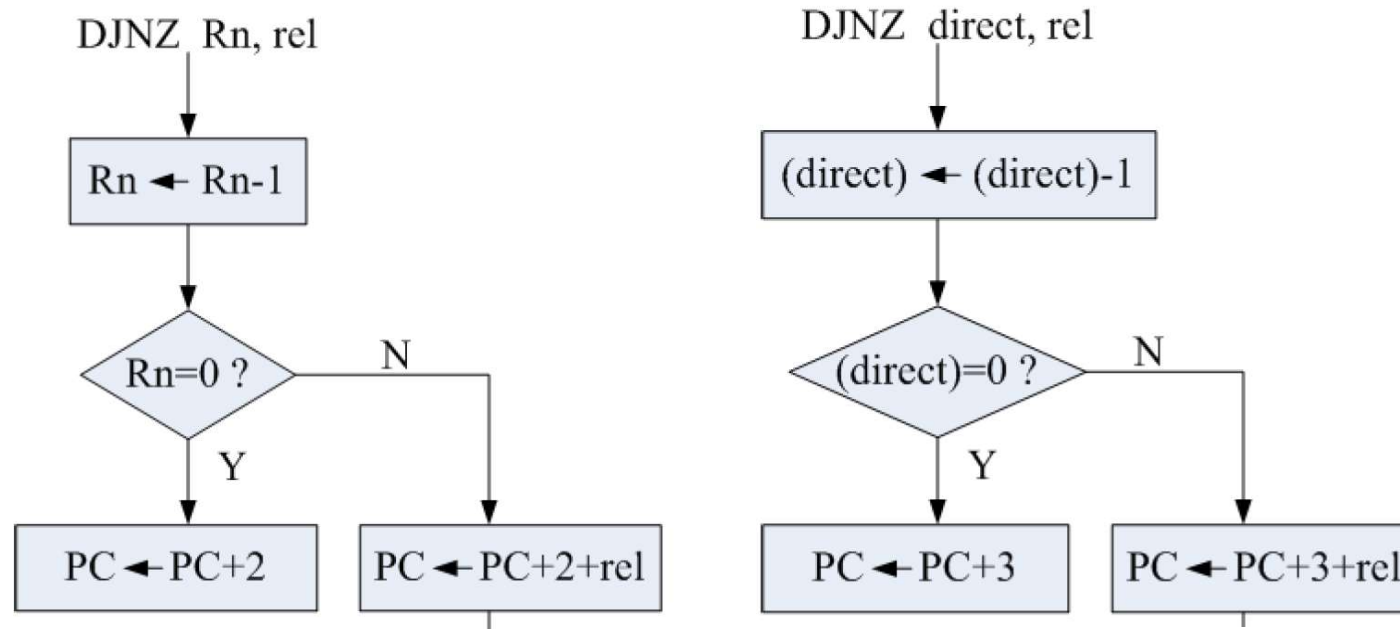
若第一操作数 **S1** 大于 或 等于 第二操作数**S2**，则  $C_Y=0$ ；  
若第一操作数 **S1**小于 第二操作数 **S2**，则 $C_Y=1$ 。

操作数**S1**，**S2**的内容保持不变！



### 3.3.4 控制转移类指令

- 指令执行示意图：



- 使用**DJNZ**指令前，应将循环次数赋值给计数器，
- 即预置到工作寄存器或片内**RAM**直接地址单元中。

### 3.3.4 控制转移类指令

- 二、调用子程序及返回指令

- 1、调用子程序指令(2条)

- LCALL** : 长调用, 可以调用**64KB**范围内的子程序

- ACALL**: 绝对调用, 限定调用**2KB**范围内的子程序

- 指令执行的操作:

- 1) 保存返回地址 (保存**16**位返回地址, 通过两次入栈操作实现)
    - 2) 执行跳转操作, 转移到子程序的入口。



### 3.3.4 控制转移类指令

**LCALL addr16** ;  $PC \leftarrow PC + 3,$   
;  $SP \leftarrow SP + 1, ((SP)) \leftarrow PC_{7 \sim 0}$   
;  $SP \leftarrow SP + 1, ((SP)) \leftarrow PC_{15 \sim 8}$   
;  $PC \leftarrow \text{addr16}$

**ACALL addr11** ;  $PC \leftarrow PC + 2,$   
;  $SP \leftarrow SP + 1, ((SP)) \leftarrow PC_{7 \sim 0}$   
;  $SP \leftarrow SP + 1, ((SP)) \leftarrow PC_{15 \sim 8}$   
;  $PC_{10 \sim 0} \leftarrow \text{addr11}$

### 3.3.4 控制转移类指令

- 2、返回指令(2条)

**RET** ;  $PC_{15\sim8} \leftarrow ((SP))$ ,  $SP \leftarrow SP-1$   
;  $PC_{7\sim0} \leftarrow ((SP))$ ,  $SP \leftarrow SP-1$

- 子程序返回指令。
- 功能：将堆栈内的返回地址送入**PC**，  
使**CPU**返回到原断点地址处，继续执行原程序。

- **RETI** ;  $PC_{15\sim8} \leftarrow ((SP))$ ,  $SP \leftarrow SP-1$   
;  $PC_{7\sim0} \leftarrow ((SP))$ ,  $SP \leftarrow SP-1$

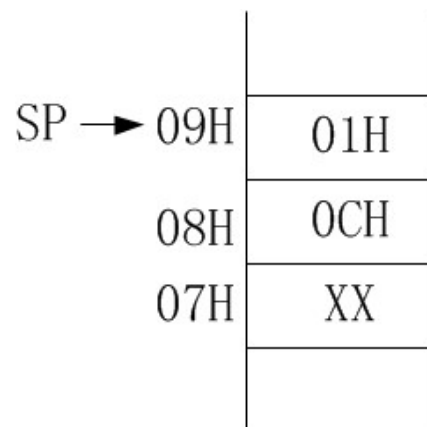
- 中断返回指令。  
除了执行**RET**指令的功能外，  
还清除内部相应的中断状态寄存器的内容。

### 3.3.4 控制转移类指令

| Loc   | Obj      |                    |
|-------|----------|--------------------|
| 0109H | 12 20 00 | <b>LCALL 2000H</b> |
| 010CH | 00       | NOP                |
| ..... |          |                    |
| 2000H | XX XX    | MOV A, 40H         |
|       | .....    |                    |
|       | 22       | RET                |

**LCALL 2000H**

;PC  $\leftarrow$  PC + 3 , PC = 010CH  
 ;SP=SP+1,SP=08H, (08H)  $\leftarrow$  PC<sub>7~0</sub>  
 ;SP=SP+1,SP=09H , (09H)  $\leftarrow$  PC<sub>15~8</sub>  
 ;PC  $\leftarrow$  add16, PC = 2000H



执行之后:

**(09H)=01H**

**(08H)=0CH**

**SP = 09H**

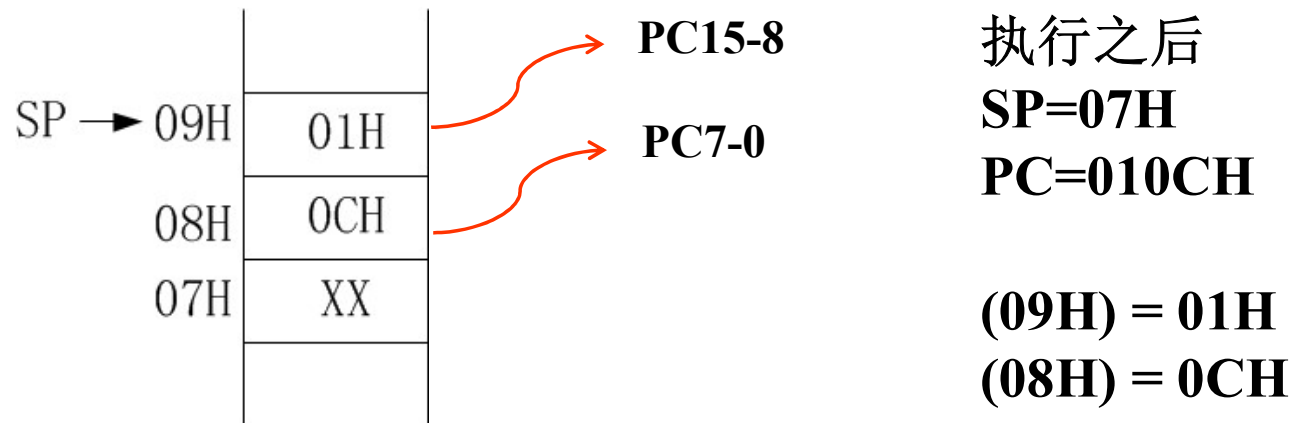
**PC = 2000H**

口诀: “低对低, 高对高”

### 3.3.4 控制转移类指令

| Loc   | Obj      |             |
|-------|----------|-------------|
| 0109H | 12 20 00 | LCALL 2000H |
| 010CH | 00       | NOP         |
| ..... |          |             |
| 2000H | XX XX    | MOV A, 40H  |
| ..... |          |             |
| 22    |          | RET         |

**RET** ;  $PC_{15\sim8} \leftarrow ((SP))$ ,  $SP \leftarrow SP-1$   
 ;  $PC_{7\sim0} \leftarrow ((SP))$ ,  $SP \leftarrow SP-1$



**09H, 08H 单元的内容保持不变!**

### 3.3.4 控制转移类指令

提示:

- **RETI**只能用于中断服务程序
- **RETI**与**RET**不能互换使用。
- 返回指令只能置于子程序或中断服务程序的末尾。

## 3.3.4 控制转移类指令

- 三、空操作指令

**NOP ; PC←PC+1**

- 单字节指令。不作任何操作(即空操作)。
- 常用于产生一个机器周期的延迟，用于时序配合。
- 或插入在程序代码中，便于设置断点，方便程序调试。

## 3.3.5 位操作类指令

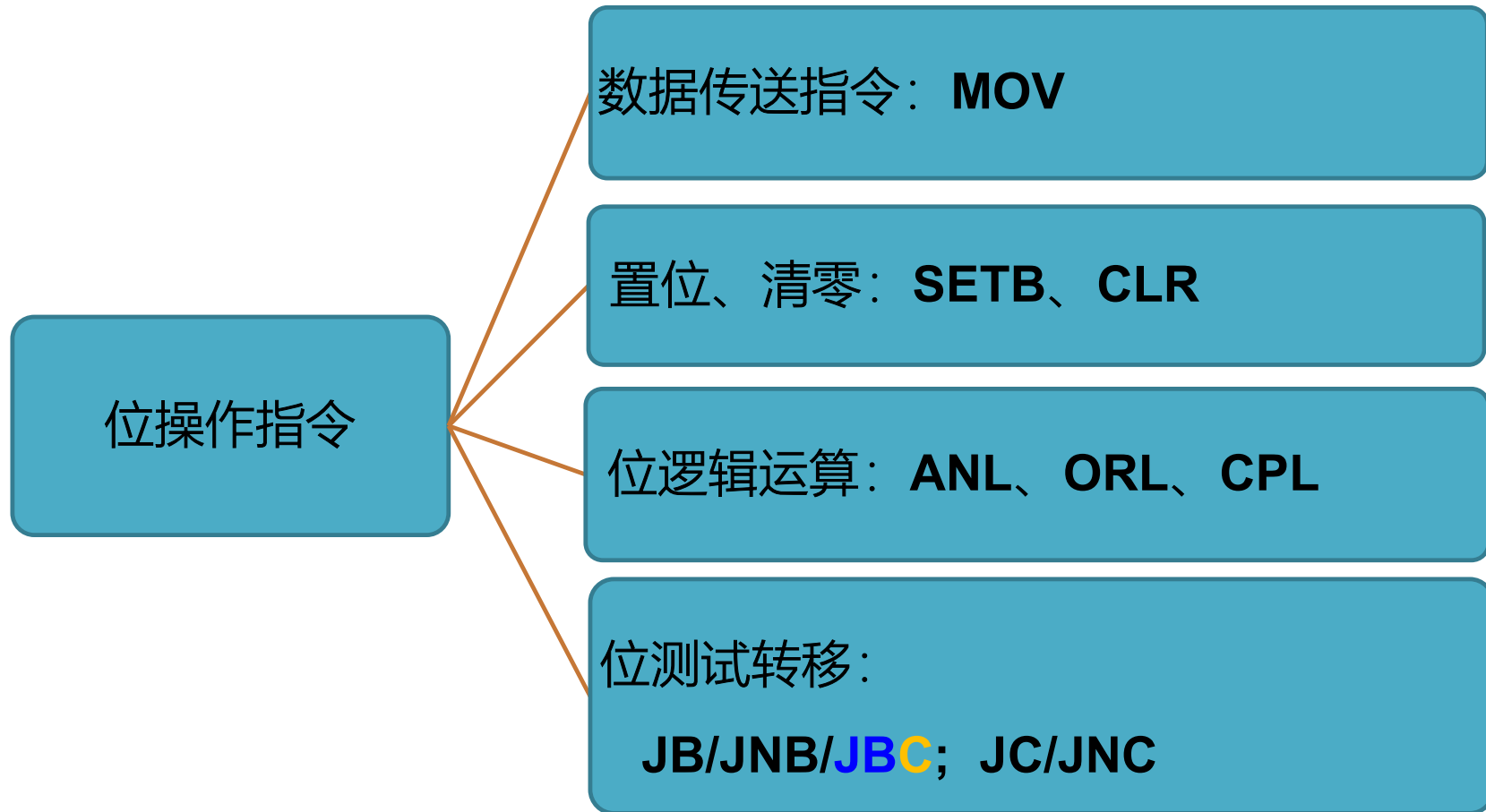
- 位操作:

- **CPU**不是以字节而是以位作为处理对象。

- 支持位寻址的空间:

- 片内**RAM**中**20H~2FH**单元, 共**16**个字节, **128**个位。
  - 特殊功能寄存器区中, 字节地址能被**8**整除的专用寄存器。

### 3.3.5 位操作类指令





### 3.3.5 位操作类指令

- 一、位数据传送指令

**MOV C, bit ;  $C \leftarrow (\text{bit})$**

**MOV bit, C ;  $(\text{bit}) \leftarrow C$**

— 位数据传送必须通过位累加器**C<sub>Y</sub>**来实现

### 3.3.5 位操作类指令

- 二、位置位、清零、取反指令

CLR C ;  $C \leftarrow 0$

CLR bit ;  $(bit) \leftarrow 0$

CPL C ;  $C \leftarrow \overline{C}$

CPL bit ;  $(bit) \leftarrow (\overline{bit})$

SETB C ;  $C \leftarrow 1$

SETB bit ;  $(bit) \leftarrow 1$

- 功能：对位累加器C<sub>Y</sub>或直接寻址的位分别进行清0、取反、置1。

## 3.3.5 位操作类指令

- 三、位逻辑运算指令

ANL C, bit ;  $C \leftarrow C \wedge (\text{bit})$

ANL C, /bit ;  $C \leftarrow C \wedge (\overline{\text{bit}})$

ORL C, bit ;  $C \leftarrow C \vee (\text{bit})$

ORL C, /bit ;  $C \leftarrow C \vee (\overline{\text{bit}})$

- 功能: 把位累加器C<sub>Y</sub>的内容及直接指定的位地址的内容分别逻辑与、逻辑或, 运算结果存于C<sub>Y</sub>中。
- "/"表示对该位地址内容取反后(不改变位地址原内容)再参与运算。

## 3.3.5 位操作类指令

- 四、位条件转移指令

**JB bit, rel**

$PC \leftarrow PC + 3$ , 若  $(bit) = 1$ , 则  $PC \leftarrow PC + rel$

**JNB bit, rel**

$PC \leftarrow PC + 3$ , 若  $(bit) = 0$ , 则  $PC \leftarrow PC + rel$

**JBC bit, rel**

$PC \leftarrow PC + 3$ , 若  $(bit) = 1$ , 则  $PC \leftarrow PC + rel$ ,  $(bit)$

$\leftarrow 0$

它们均是3字节指令。

### 3.3.5 位操作类指令

#### CY位测试指令

**JC** rel; PC=PC + 2 , if CY = 1, then PC = PC + rel

**JNC** rel; PC=PC + 2 , if CY = 0, then PC = PC + rel

- 例：测试**Cy**的状态，实现二分支。

```
MOV    A,    20H
ADD    A,    21H
JNC     HERE    ; 当 Cy = 0 时，程序跳转到HERE
MOV    22H,   #1    ; 保存和的高位
HERE:  SJMP   HERE
```

### 3.3.5 位操作类指令

#### 位操作指令典型应用

1. 修改位状态 (**SETB**、**CLR**、**CPL**)
2. 利用位测试指令实现二分支结构，或循环结构。

## 3.4 汇编指令系统总结

- 指令对PSW的影响

Instructions that Affect Flag Settings<sup>(1)</sup>

| Instruction | Flag |    |    | Instruction | Flag |    |    |
|-------------|------|----|----|-------------|------|----|----|
|             | C    | OV | AC |             | C    | OV | AC |
| ADD         | X    | X  | X  | CLR C       | 0    |    |    |
| ADDC        | X    | X  | X  | CPL C       | X    |    |    |
| SUBB        | X    | X  | X  | ANL C,bit   | X    |    |    |
| MUL         | 0    | X  |    | ANL C,/bit  | X    |    |    |
| DIV         | 0    | X  |    | ORL C,bit   | X    |    |    |
| DA          | X    |    |    | ORL C,/bit  | X    |    |    |
| RRC         | X    |    |    | MOV C,bit   | X    |    |    |
| RLC         | X    |    |    | CJNE        | X    |    |    |
| SETB C      | 1    |    |    |             |      |    |    |

这里：X 代表影响标志位。

当指令的第一操作数是A时，会影响P标志。依据A的内容，按偶校验原则，刷新P标志。

## 3.4 汇编指令系统总结

- **A特有指令**

**DA A**

**CLR A** (字节清零指令)

**CPL A** (字节取反指令)

**RL A**

**RR A**

**RLC A**

**RRC A**

**SWAP A**

**ADD** (第一操作数必须是 **A**)

**ADDC** (第一操作数必须是 **A**)

**SUBB** (第一操作数必须是 **A**)

**MUL AB**

**DIV AB**



## 3.4 汇编指令系统总结

- 指令执行周期

指令按执行周期分为单周期、双周期、四周期指令。

双周期指令：

控制转移类指令（全部）

**MOVC, MOVX**

**PUSH、POP**

其它（.....）

四周期指令：**MUL、DIV**

# 汇编语言伪指令

## ■ 伪指令的作用和使用方法

### ● 伪指令的作用

写在源文件中，用于控制汇编过程的命令。

如设置程序或数据存储区的地址、定义符号、判断程序是否结束等。

没有对应的机器码，它是不可执行的指令。

# 汇编语言伪指令

## ● 伪指令种类：

ORG：指定语句行装载的起始地址，可以在同一文件中出现多次。

END：指示语句行到此结束，一般出现在程序行结束以前。

EQU：赋值指令，用于定义常数，或地址。

等同于C中的 # define语句。

DATA：定义字节地址。

BIT：定义位符号地址。

DB：定义字节数据。用于给代码空间的存储单元进行初始化、赋值，或定义表格。

DW：定义字数据，即两个字节。用于给代码空间的存储单元进行初始化、赋值，或定义表格。

DS：预留若干个存储单元，等同于C中的malloc函数。

## 汇编语言伪指令

### EQU (Equate) 赋值指令。

将操作数段中的地址或数据赋值给标号。

赋值后的标号，其值在整个程序中不改变，可多次使用。

命令格式： 标号 EQU 数或汇编符号

```
例：  COUNT      EQU  16H           ; COUNT = 16H
      ADDR       EQU  3000H          ; ADDR = 3000H
      MOV A,     # COUNT             ; A = 16H
```

标号与EQU之间不能用“:”来作分隔符。

# 汇编语言伪指令

## DATA 数据地址赋值指令

将数据地址或代码地址赋予所规定的标号。

命令格式为：    字符名称    DATA    表达式

例如，    MN    DATA    10H

汇编后，MN的值为10H。

DATA指令在程序中常用来定义存放数据的单元字节地址。

# 汇编语言伪指令

## BIT 位地址符号命令

将位地址赋予所规定的字符名称，常用于定义位符号地址。

命令格式为：            字符名称 BIT 位地址

例如，            AA BIT P1.0

BB BIT P2.0

# 汇编语言伪指令

## DB (Define Byte) 定义字节命令

定义字节数据。它的作用是从指定的地址单元开始，定义数据或ASCII码字符，常用于定义数据常数表。

命令格式： **[标号:] DB 字节常数表**

例如：

```
ORG 2000H  
TAB: DB 14H, 26, 'A'  
      DB 0AFH, 'BC'
```

汇编结果：

```
(2000H) = 14H      (2001H) = 1AH  
(2002H) = 41H      (2003H) = AFH  
(2004H) = 42H      (2005H) = 43H
```

|       |    |
|-------|----|
| 2000H | 14 |
| 2001H | 1A |
| 2002H | 41 |
| 2003H | AF |
| 2004H | 42 |
| 2005H | 43 |

# 汇编语言伪指令

## DW (Define Word) 定义字命令

定义16位数据。从指定的地址单元开始，定义若干个字常数，常用于定义地址表。

命令格式为：[标号:] DW 字常数表

例如，  
ORG 2000H  
TAB: DW 7423H, 00ABH, 20

汇编结果：  
(2000H) = 74H      (2001H) = 23H  
(2002H) = 00H      (2003H) = ABH  
(2004H) = 00H      (2005H) = 14H

|       |    |
|-------|----|
| 2000H | 74 |
| 2001H | 23 |
| 2002H | 00 |
| 2003H | AB |
| 2004H | 00 |
| 2005H | 14 |

**提示：** 一个字占两个存储单元，其中高字节数存入低位地址，低字节数存入高位地址，即顺序存放。



# 汇编语言伪指令

## DS (Define Store) 定义存储区

定义存储区。从指定的地址开始，保留一定数量的内存单元，以备程序使用，其区域的大小由指令的操作数确定。

命令格式：     [标号:]   DS     数值

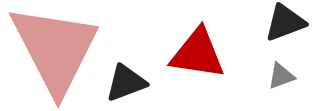
例如：                 ORG   1000H  
                          DS     5  
                          DB     23H

|       |     |
|-------|-----|
| 1000H | --  |
| 1001H | --  |
| 1002H | --  |
| 1003H | --  |
| 1004H | --  |
| 1005H | 23H |

汇编结果：

从地址1000H开始，保留5个字节的内存单元，而(1005H)=23H

# 4

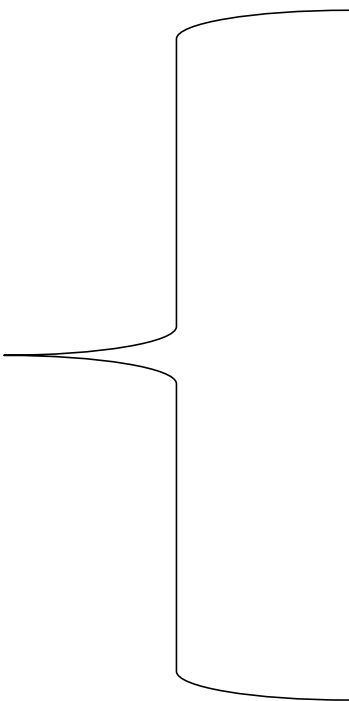


## Chapter

# 汇编语言程序设计



## 4 汇编语言程序设计



顺序程序

分支程序

循环程序

子程序

## 4.3 循环程序设计

**例：**条件控制的循环程序

将内部RAM 30H中的无符号数转换为十进制数，结果放入R6和R7中。

**要点：**

- 30H中的值，转化化10进制后，最大为3位（即到百位），需要3个中间结果寄存器
- 循环除以10，才能得到个位、十位、百位
- 最后将3个中间结果寄存器，以压缩BCD码的方式存入R6、R7

## 4.3 循环程序设计

```
MOV    31H, #0      ;31H~33H 3单元清零, 百位
MOV    32H, #0      ;十位
MOV    33H, #0      ;个位
MOV    R0,  #33H    ;设置间址寄存器
MOV    A,    30H    ;无符号数送A
LOOP:  MOV    B,    #10 ;置除数10
      CJNE   A,    #0, NEZ ;被除数为0?
      MOV    A,    32H ;为0, 将31H~33H中的 BCD数
      SWAP   A        ;拼为压缩BCD数放入R6和R7中, 10进
                        ;制数只占4bit位
      ORL    A,    33H ;ORL操作有拼位作用
      MOV    R7,  A
      MOV    R6,  31H
      SJMP   $
NEZ:   DIV    AB      ;被除数不为0, 除以10
      MOV    @R0, B   ;存余数, 逐渐存入33H~31H
      DEC    R0       ;修改间址指针
      SJMP   LOOP
      END
```

## 4.3 循环程序设计

**例：**双重控制的循环程序

把内部RAM中起始地址为30H的字节数据串传送到外部RAM1000H为首地址的区域，直到发现 ‘\$’字符的ASCII码为止。数据串的最大长度为32个字节。

**要点：**

□这是计数(32个)控制和条件控制('\$')双重控制的循环程序。

## 4.3 循环程序设计

```
MOV    R0,    #30H      ;内部RAM字节串间址指针
MOV    DPTR,  #1000H    ;外部RAM字节串间址指针
MOV    R7,    #32       ;设置计数循环计数器
LOOP:  CJNE   @R0,  #'$', ISO ;是 '$' 吗? 不是转移去ISO
      SJMP   $          ;是, 结束
ISO:   MOV    A,    @R0   ;从内部RAM中取数
      MOVX   @DPTR, A    ;送外部RAM, 到外部经过A
      INC    R0          ;修改内部RAM间址指针
      INC    DPTR        ;修改外部RAM间址指针
      DJNZ   R7,    LOOP ;已传送32个吗? 没有, 循环
      SJMP   $          ;传送32个, 结束
      END
```

## 4.3 循环程序设计

**例：**编制将连续存放在内部RAM20H为首地址存储器区域中的n个无符号字节数据的排序的程序。

**要点：**

□ 排序问题可以采取逐一比较法和两两比较法。



## 4.3 循环程序设计

逐一比较法:

```
MOV    R0, #20H    ;R0指向数据区
MOV    R7, #19      ;置外循环计数器R7, (n=20)
OUT:   MOV    R6, R7    ;置内循环计数器R6
        PUSH  00H      ;0组的R0进栈
        MOV   A,  @R0    ;取内循环环间址指针
IN:     INC    R0
        MOV   B,  @R0
        CJNE  A,  B, $+3 ;比较A≥@R0?
        JNC   NXCHG     ;大于等于不交换
        XCH   A,  @R0    ;小于交换
NXCHG: DJNZ   R6, IN      ;判断内循环是否结束
        POP   00H      ;内循环结束R0出栈
        MOV   @R0, A     ;存内循环的最大值
        INC   R0        ;修改外循环间址指针
        DJNZ  R7, OUT    ;判断外循环是否结束?
        SJMP  $
END
```

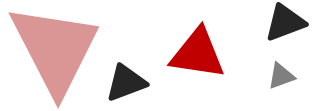
## 4.3 循环程序设计

### 两两比较法

```

        CLR    F0                ;清交换标志
        MOV    R7, #19          ;设置外循环计数器
OUT:     MOV    R0, #20H         ;R0指向数据区
        ; //MOV R7, #19 ;设置外循环计数器
        MOV    R6, R7
IN:      MOV    A, @R0           ;取一数据到A
        INC    R0
        MOV    B, @R0           ;取下一单元数据到B
        CJNE   A, B, $+3        ;A≥@R0
        JNC    NEXCHG           ;大于等于不交换
        XCH    A, @R0           ;小于交换
        DEC    R0
        MOV    @R0, A
        INC    R0
        SETB   F0               ;置交换标志
NEXCHG: DJNZ   R6, IN            ;内循环结束?
        DEC    R7               ;下次的内循环少一次
        JBC    F0, OUT          ;交换发生继续外循环,并清交换标志
        SJMP   $                ;未交换结束循环
        END
```

# 7



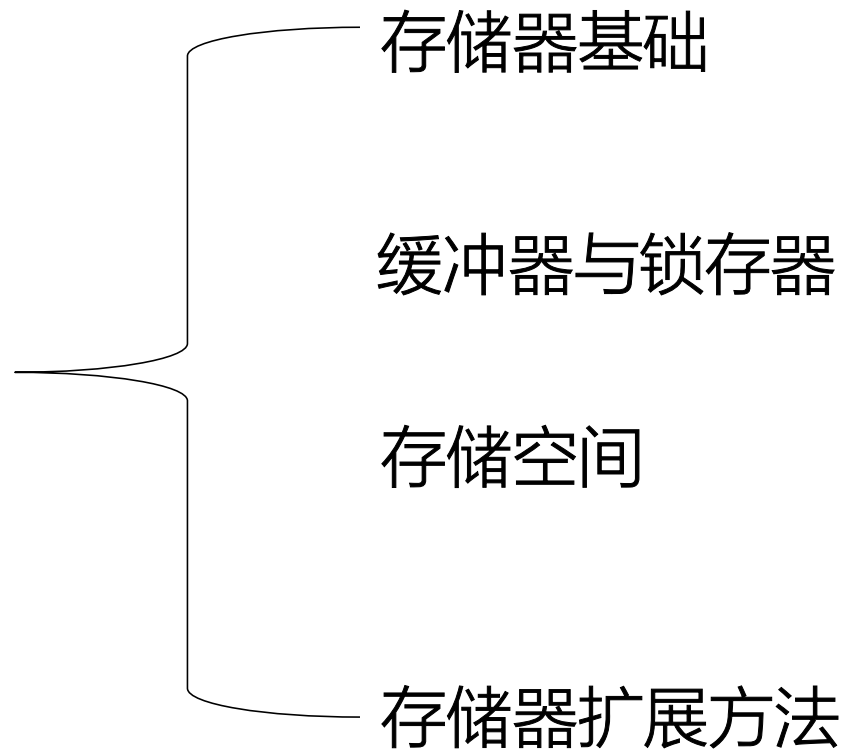
## Chapter

存储器





[ 7 存储器



## 7 存储器基础

### 一、单片机系统扩展的工作特点

➤ 需要总线维持信号较长的设备，要自行添加锁存器。（输出锁存）

——原因：总线没有记忆能力

➤ 设备与总线间加三态门保持输入缓冲。（输入缓冲）

——原因：同一时刻只允许一台设备发送信号

### 二、存储器类型

ROM：2716（2K\*8Bit）、2764（8K\*8Bit）、27256（32K\*8Bit）

RAM：6116（2K\*8Bit）、6264（8K\*8Bit）、62256（32K\*8Bit）

### 三、存储器主要性能指标

存储容量=字数\*字长；

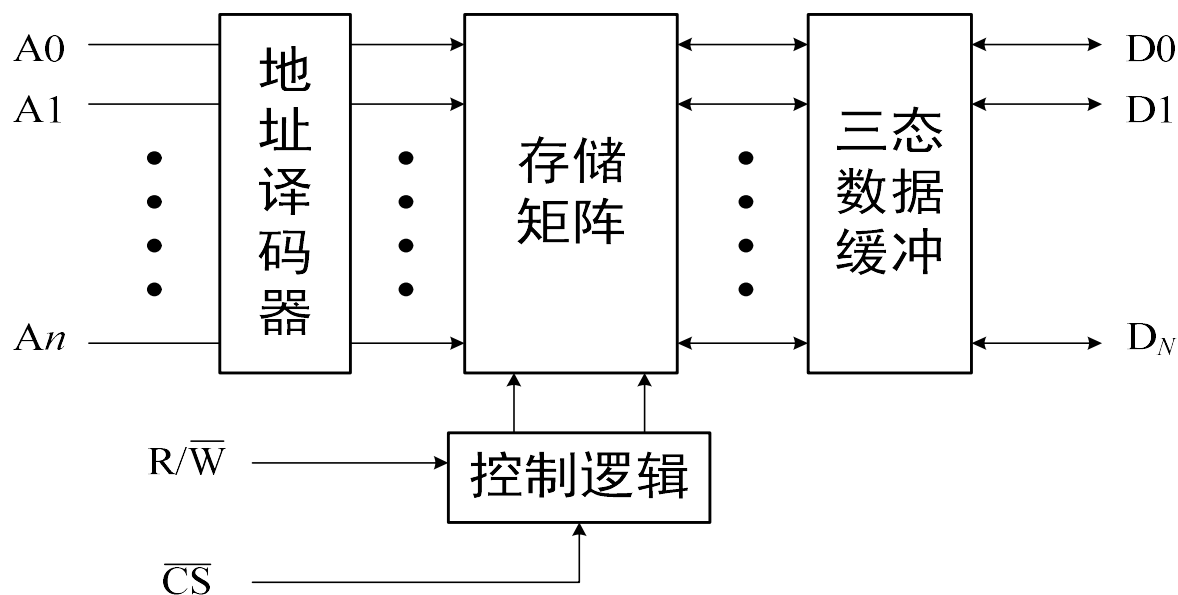
存取速度：从CPU给出有效地存储器地址到存储器输入或输出有效数据所需要的时间。

## 7 存储器基础

### 四、半导体存储芯片结构

➤ 半导体存储器芯片由存储矩阵、地址译码器、控制逻辑和三态数据缓冲寄存器组成，是大量存储元件的有机结合。存储元件则是由能存储一位二进制代码的物理器件构成。

➤ 将存储矩阵中的全部存储单元赋予单元地址，由芯片内部的地址译码器实现按地址选择对应的存储单元。在CPU及其接口电路送来的芯片选择信号CS和读写控制信号R/W的配合下，单方面打开三态缓冲器，将该存储单元中的代码进行读或写操作。在不进行读或写操作时，芯片选择信号无效，控制逻辑使三态缓冲器处于高阻组态，存储矩阵与数据线脱开。



## 7 存储器基础

### 五、6116芯片工作方式

- 写入方式。其条件是:CE =0, WE=0, OE=1。操作结果是D0-D7上的内容输入到A0-A10所指定的存储单元中。
- 读出方式。其条件是:CE= 0, WE=1, OE=0。操作结果是A0-A10所指定的存储单元内容输出到D0-D7上。
- 低功耗维持方式, 这是一种非工作方式, 当CE=1时, 芯片处于这一方式。此时, 器件电流仅为20uA左右, 为系统断电时用电池保持RAM的内容提供了可能性。

| /CS | /WE | /OE | Mode            | I/O Operation |
|-----|-----|-----|-----------------|---------------|
| H   | X   | X   | Standby         | High-Z        |
| L   | H   | H   | Output Disabled | High-Z        |
| L   | H   | L   | Read            | Data Out      |
| L   | L   | X   | Write           | Data In       |

## 7 存储器基础

### 六、2716芯片工作方式——读方式

- 这是2716芯片通常使用的方式，此时两个电源引脚VCC和VPP都接至+5V。
- 当从2716芯片的某个单元中读数据时，先通过A0-A10接收来自CPU的地址信号，然后使CE和OE都有效，于是经过一段延迟时间，D0-D7便送出该地址所指定的存储单元的内容。

未选中

- 在OE=1时，不论CE状态如何，2716芯片均未选中，因此D0-D7呈高阻状态。

| 方式 \ 引脚 | V <sub>CC</sub> | V <sub>PP</sub> | $\overline{\text{CE}}$ | $\overline{\text{OE}}$ | D <sub>0</sub> ~D <sub>7</sub> |
|---------|-----------------|-----------------|------------------------|------------------------|--------------------------------|
| 读       | +5V             | +5V             | 低                      | 低                      | 输出                             |
| 未选中     | +5V             | +5V             | ×                      | 高                      | 高阻                             |
| 等待      | +5V             | +5V             | 高                      | ×                      | 高阻                             |
| 编程      | +5V             | +25V            | 正脉冲                    | 高                      | 输入                             |
| 编程检查    | +5V             | +25V            | 低                      | 低                      | 输出                             |
| 编程禁止    | +5V             | +25V            | 低                      | 高                      | 高阻                             |



## 7 缓冲器与锁存器

### 一、缓冲器

CPU总线的缓冲，增强其带动负载能力。

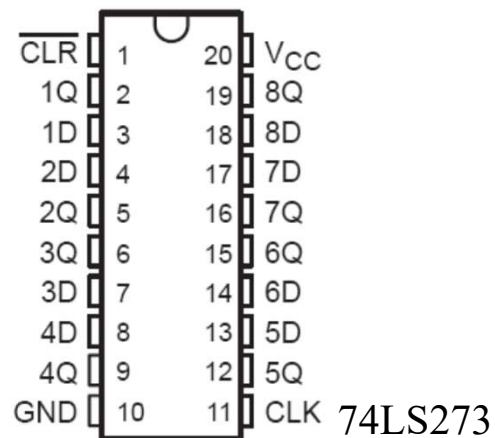
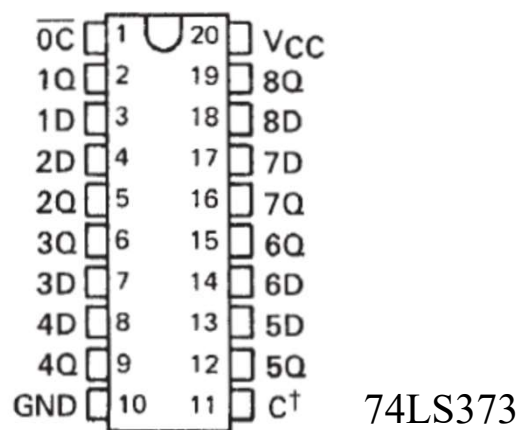
74LS244单向缓冲器，用于地址总线/控制总线缓冲

74LS245双向缓冲器，用于数据总线缓冲

### 二、锁存器

利用锁存器，把地址信号从地址/数据总线中分离

74LS373电平触发，有三态门缓冲，存在高阻态；74LS273上升沿触发



## 7 存储空间

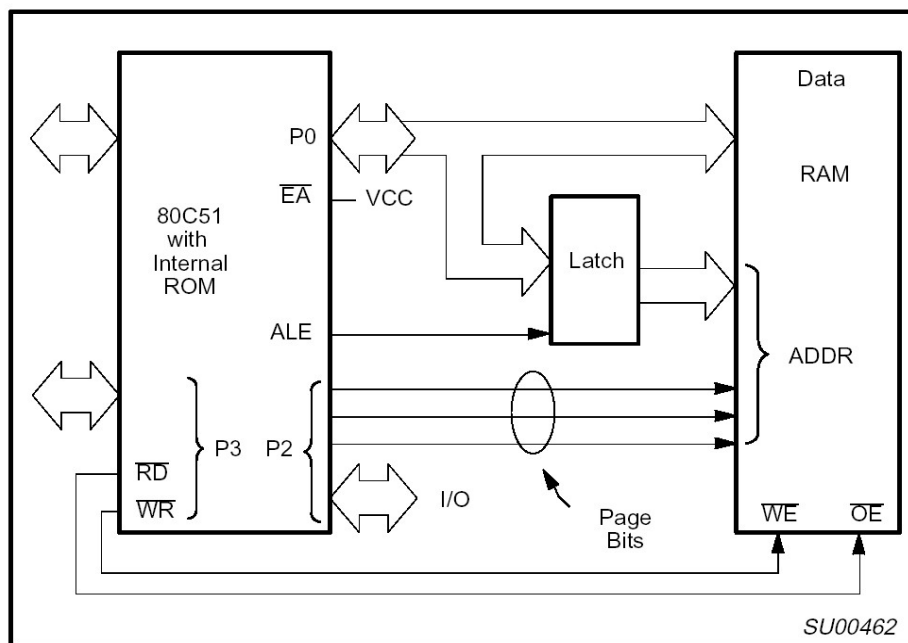
- 从物理上可以分为片内、片外两个区域；
- 片外区域在逻辑上可以分为ROM和RAM两个区域；
- 访问这两个逻辑区域所用的控制信号和访问指令都是不一样的；
- 外部设备在逻辑上一般划归RAM区域，通过访问RAM的方式来访问外部设备；
- 片外存储器容量扩展最大64KB；
- 程序存储器和数据存储器数据空间不同，可以同时扩展64KB；
- IO口与数据空间统一编址。

| 空间类型  | 对象                       | 控制信号      | 访问指令 |
|-------|--------------------------|-----------|------|
| 代码空间  | ROM ,E <sup>2</sup> PROM | /PSEN     | MOVC |
| 数据空间  | RAM                      | /RD , /WR | MOVX |
| IO 空间 | A/D D/A并行口等              | /RD , /WR | MOVX |

## 7 存储器扩展方法

### □ 数据存储器基本扩展方法

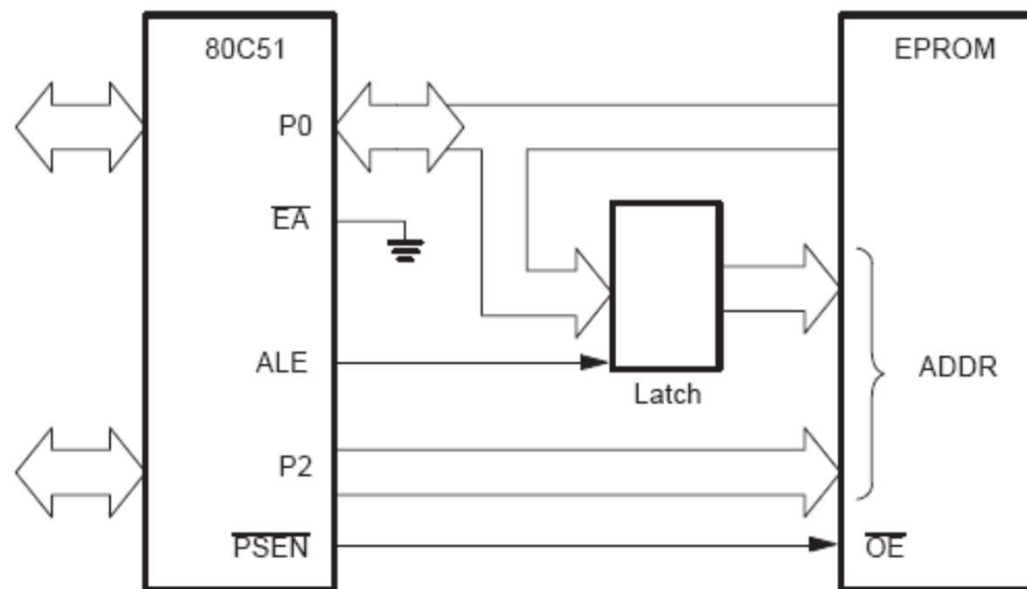
- 扩展数据存储器要将存储器芯片的OE和WE分别和单片机系统的RD和WR相连。可采用EEPROM只读存储器、静态RAM和动态RAM三类芯片。



## 7 存储器扩展方法

### □ 程序存储器基本扩展方法

- 扩展程序存储器主要是将存储器芯片的OE与单片机系统的PSEN相连。
- 扩展程序存储器可以采用只读存储器的3种芯片。

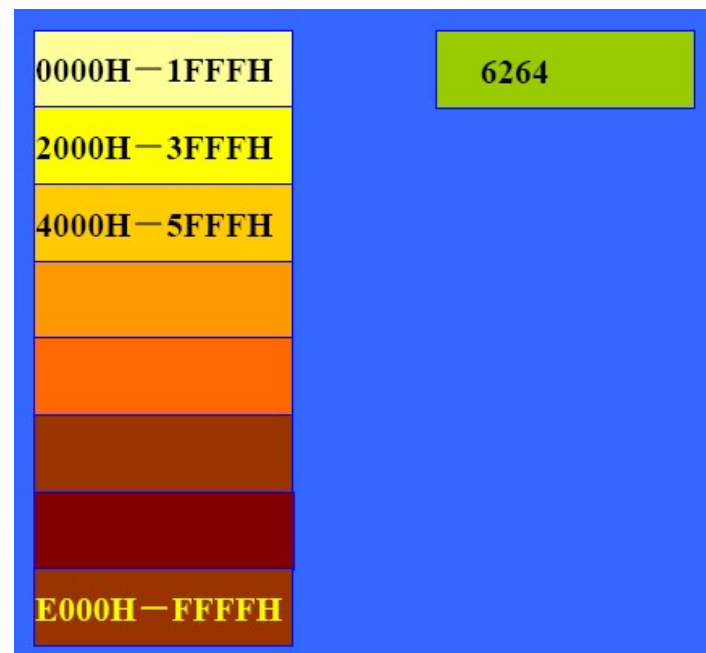


## 7 存储器扩展方法

### 7.2.1 存储空间分配

#### □ 存储空间分配

- 存储空间是计算机资源之一分配存储空间：指确定存储芯片或接口芯片地址（范围）的过程。
- 如6264 芯片，内含8KB，即8192 个存储单元。
- 给6264 分配存储空间的含义是确定其每个存储单元的地址。



## 7 存储器扩展方法

### 7.2.2 片选信号的产生方法

#### □ 片选信号的产生方法

- 片选信号 ( $/CS$ ,  $/CE$ ) 的作用:
- 决定每个器件是否具有使用总线的权利。
- 片选信号的产生方法: 直接接地法、线选法、译码法

##### 1、直接接地法

- 地址线  $A_{12} - A_0$ : 用于片内译码, 达到选择某个特定单元的目的。
- 地址线  $A_{15}$ ,  $A_{14}$ ,  $A_{13}$  为无关态。



## 7 存储器扩展方法

- 6242 共有8 块存储地址，占用了全部数据空间  
0000H - 1FFFFH  
2000H - 3FFFFH  
4000H - 5FFFFH  
6000H - 7FFFFH  
8000H - 9FFFFH  
A000H - BFFFFH  
C000H - DFFFFH  
E000H - FFFFFH
- 6242 共有8 块存储地址，即存在重复地址，它占用了全部数据空间！

## 7.2 存储空间分配和片选信号产生方法

### 2、线选法

□ 直接利用单根地址线作为片选信号。

□ 例：采用线选法扩展两片8KBROM以及8KBRAM

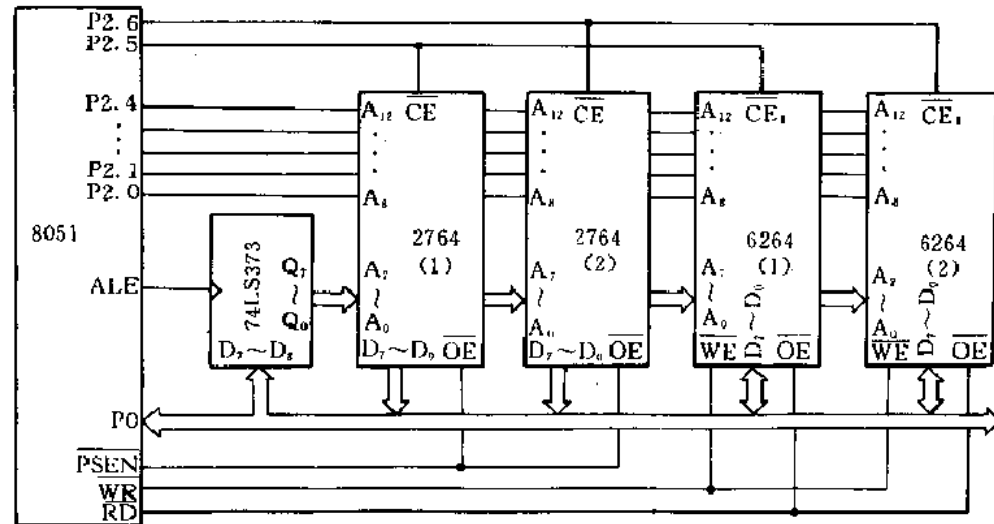
□ 6264和2764都是8KX8的存储器，它们都有13根地址线A0-A12；

□ 用剩余3根地址线

A13-A15分别作片选信号；

□ A13(P2.5)连2764(1)和6264 (1)的片选端；

□ A14(P2.6)连2764(2)和6264(2)的片选端。





## 7.2 存储空间分配和片选信号产生方法

- 访问6264 (1) 的地址约束条件为：

|            |            |            |
|------------|------------|------------|
| A15 (P2.7) | A14 (P2.6) | A13 (P2.5) |
| X          | X          | 0          |

- 由于有两根地址线未使用，故可选择的地址空间为：

0000H - 1FFFH, 4000H - 5FFFF, 8000H - 9FFFH, C000H - DFFFH

- 这四个存储区是重叠的，例如，地址0000H, 4000H, 8000H, C000H都对应6264 (1) 中的同一个地址单元，即地址重叠了，其他几片与此类似。
- 所以用线选法实现片选，其存储单元地址不是唯一的。

## 7.2 存储空间分配和片选信号产生方法

- 访问6264 (2) 的地址约束条件为：

|            |            |            |
|------------|------------|------------|
| A15 (P2.7) | A14 (P2.6) | A13 (P2.5) |
| X          | 0          | X          |

- 由于有两根地址线未使用，故可选择的地址空间为：

0000H - 1FFFH, 2000H - 3FFFF, 8000H - 9FFFH, A000H - BFFFH

- 可见对于两片6264来说，地址空间0000H - 1FFFH和8000H - 9FFFH是重叠的，因此不能使用这一范围的地址来访问片外RAM，否则会出现两个6264都被选中的情况，会造成逻辑错误。
- 片外2764的地址空间情况与6264相同。

## 7.2 存储空间分配和片选信号产生方法

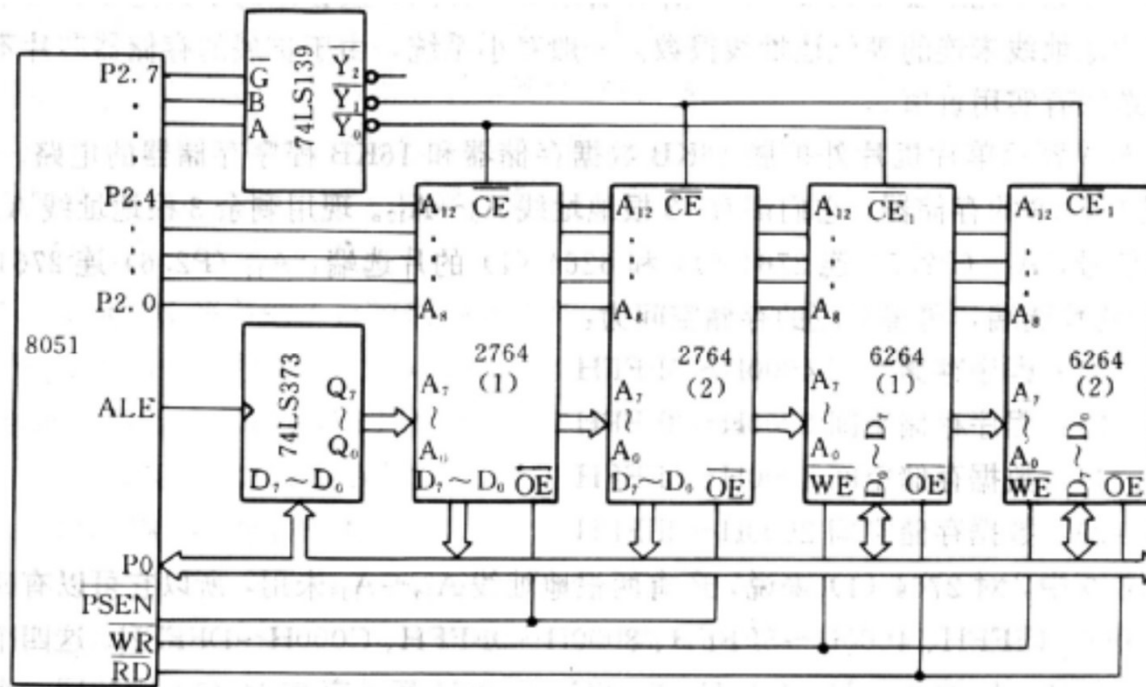
### 3、译码法

- 多根地址线经过译码器、简单逻辑电路、可编程逻辑阵列处理后产生片选信号。
- 地址译码法又有部分译码和全译码两种方式。

## 7.2 存储空间分配和片选信号产生方法

### □全译码

- 除存储器芯片所用地址线与CPU的地址线对应相连外，未用的地址线全部参加译码，通过地址译码器产生存储器的片选信号。



## 7.2 存储空间分配和片选信号产生方法

- 访问6264 (1) 的地址约束条件为:  $\overline{Y_0} = \overline{Y_1} = \overline{Y_2} = 0$

A15 (P2.7)      A14 (P2.6)      A13 (P2.5)

G

B

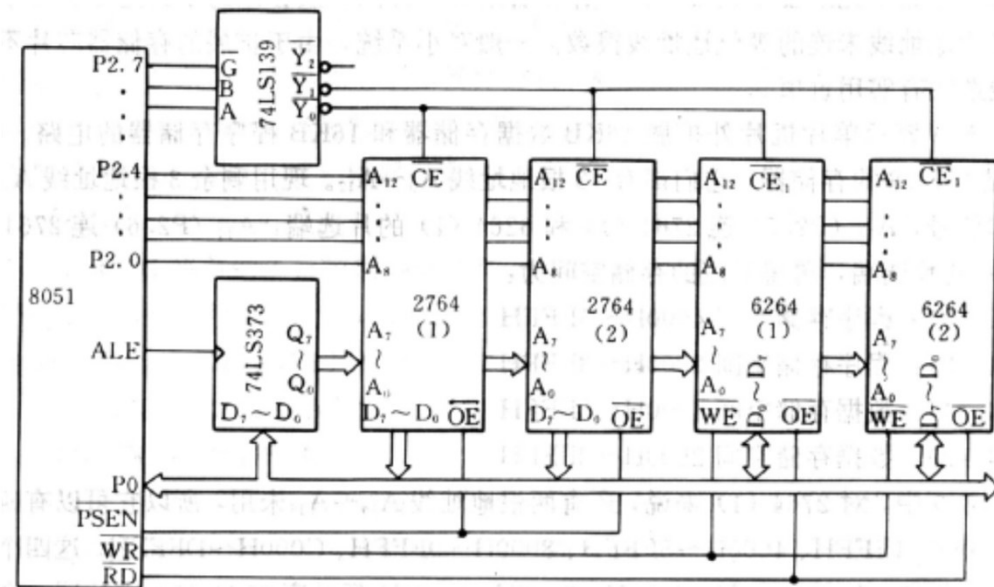
A

0

0

0

- 6264(1)的地址范围为0000H - 1FFFH



## 7.2 存储空间分配和片选信号产生方法

- 访问6264 (2) 的地址约束条件为:  $\overline{Y_0} = \overline{Y_1}$

A15 (P2.7)      A14 (P2.6)      A13 (P2.5)

G

B

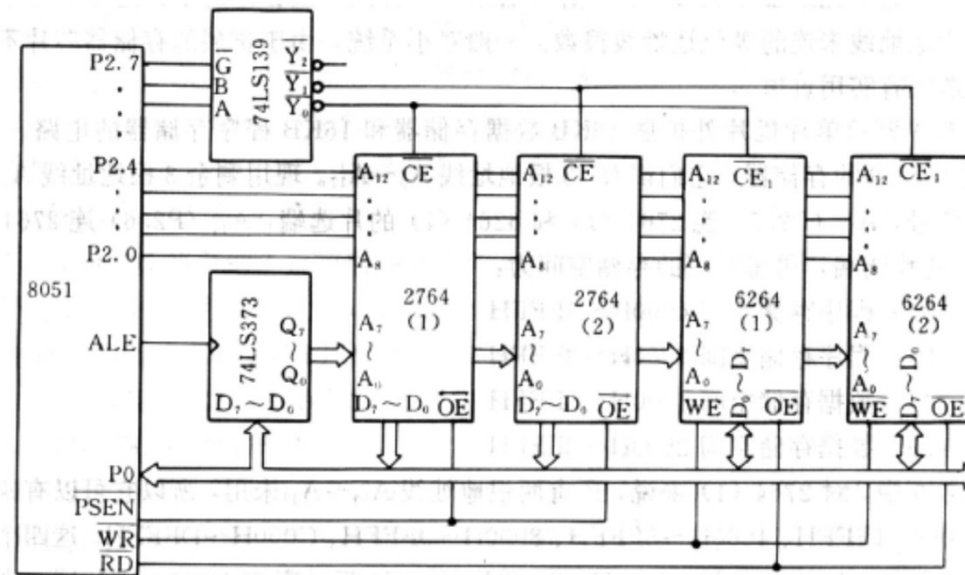
A

0

0

1

- 6264(2)的地址范围为2000H - 3FFFH
- 6264(1)和6264(2)的地址空间都是唯一的, 不存在重复地址。

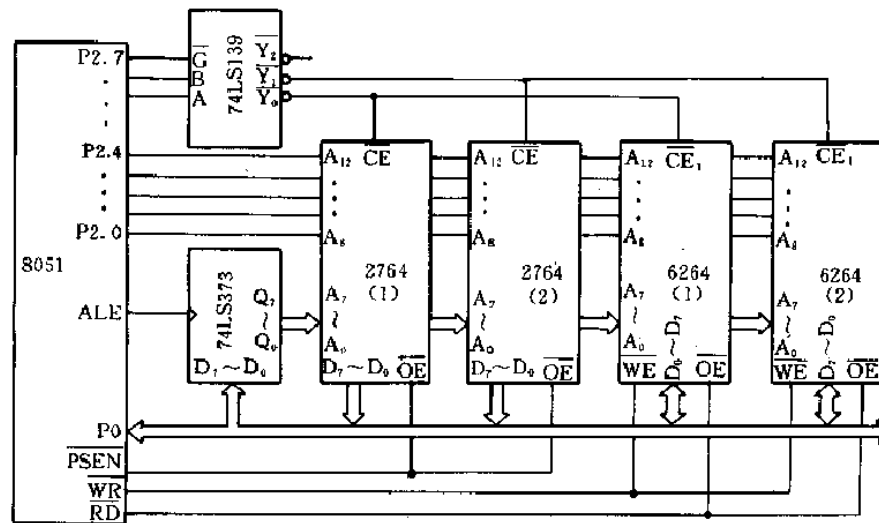


## 7.2 存储空间分配和片选信号产生方法

- 16 根地址线中低13 位（A0—A12）用于片内译码，
- 高3 位（A13、A14、A15）用于片选译码。
- 全部地址线参与了译码，6264 不再存在重复地址！

## 7.2 存储空间分配和片选信号产生方法

- 如图所示的片外存储器扩展方法，各存储器的地址空间为：
- 2764(1): 程序存储空间0000H-1FFFH (/Y0)
- 2764(2): 程序存储空间2000H-3FFFH (/Y1)
- 6264(1): 数据存储空间0000H-1FFFH (/Y0)
- 6264(2): 数据存储空间2000H-3FFFH (/Y1)



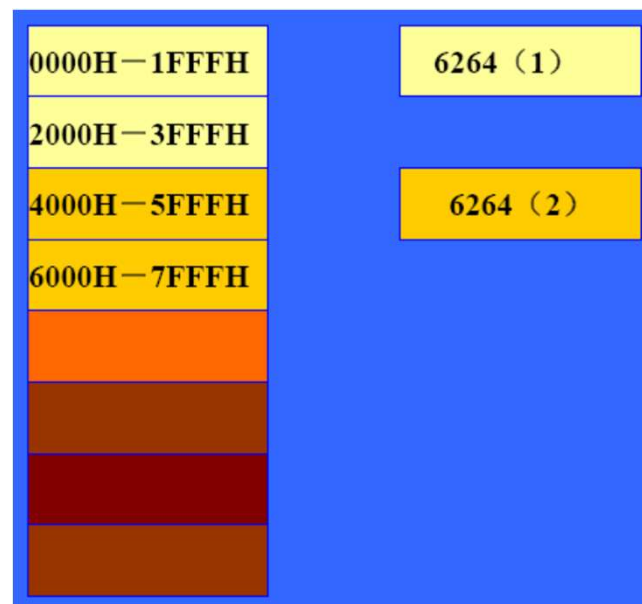


## 7.2 存储空间分配和片选信号产生方法

- 部分译码

- 部分译码是指，未用的高位地址线部分参加译码，其译码输出分别连到不同的片选端。
- 情况1：采用A15、A14参与译码，A13未使用：

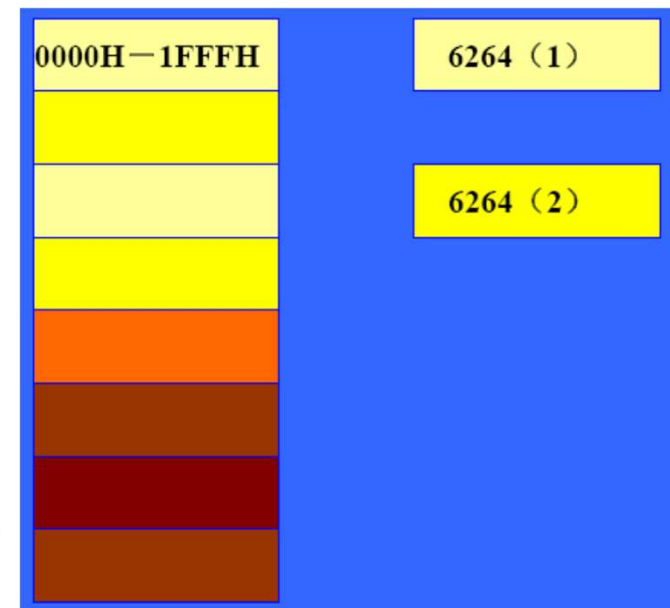
|        |                                                            |                                                            |
|--------|------------------------------------------------------------|------------------------------------------------------------|
| 芯片     | 6264 (1)                                                   | 6264 (2)                                                   |
| /CE 控制 | /CE=A15+A14                                                | /CE=A15+/A14                                               |
| 约束条件   | A15=0, A14=0                                               | A15=0, A14=1                                               |
| 地址空间   | A15-A13: 000<br>0000H-1FFFH<br>A15-A13: 001<br>2000H-3FFFH | A15-A13: 010<br>4000H-5FFFH<br>A15-A13: 011<br>6000H-7FFFH |
| 特点     | 1. 存在重复地址      2. 重复地址连续                                   |                                                            |



## 7.2 存储空间分配和片选信号产生方法

➤情况2：采用A15、A13参与译码，A14未使用：

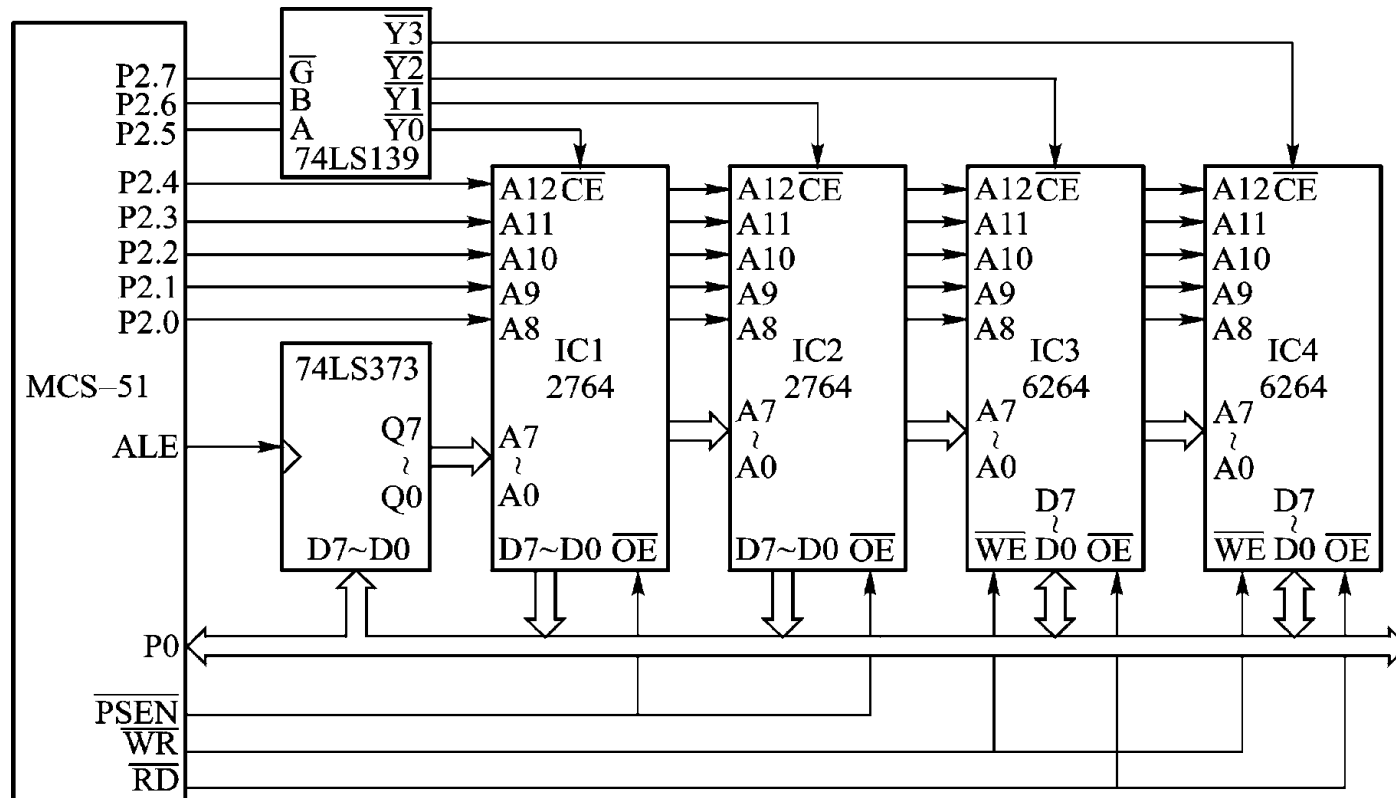
|        |                                                            |                                                            |
|--------|------------------------------------------------------------|------------------------------------------------------------|
| 芯片     | 6264 (1)                                                   | 6264 (2)                                                   |
| /CE 控制 | /CE=A15+A13                                                | /CE=A15+/A13                                               |
| 约束条件   | A15=0, A13=0                                               | A15=0, A13=1                                               |
| 地址空间   | A15—A13: 000<br>0000H—1FFFH<br>A15—A13: 010<br>4000H—5FFFH | A15—A13: 001<br>2000H—3FFFH<br>A15—A13: 011<br>6000H—7FFFH |
| 特点     | 1. 存在重复地址      2.重复地址不连续                                   |                                                            |



## 7.2 存储空间分配和片选信号产生方法

- **线选法**：简单，芯片之间的地址不连续，存在重叠区；
- **译码法**：
  - 略复杂；
  - 部分译码存在重复地址；
  - 全译码不存在重复地址，存储空间地址资源利用率高。
- **使用方法**：
  - 多采用部分译码方法；
  - **首选高位地址进行译码**；
  - 多采用简单逻辑门电路，和可编程器件（CPLD）实现。

例 采用译码器法扩展2片8KB EPROM，2片8KB RAM。  
 EPROM选用2764，RAM选用6264。共扩展4片芯片。扩展  
 接口电路见图。



各存储器地址范围如下：

| 芯片  | 地址范围        |
|-----|-------------|
| IC4 | 6000H~7FFFH |
| IC3 | 4000H~5FFFH |
| IC2 | 2000H~3FFFH |
| IC1 | 0000H~1FFFH |

可见译码法进行地址分配，各芯片地址空间是连续的。

## 外扩存储器电路的工作原理及软件设计

1. 单片机片外程序区读指令过程
2. 单片机片外数据区读写数据过程

例如，把片外1000H单元的数送到片内RAM 50H单元，程序如下：

```
MOV DPTR, #1000H
MOVX A, @DPTR
MOV 50H, A
```

例如，把片内50H单元的数据送到片外1000H单元中，程序如下：

```
MOV A, 50H
MOV DPTR, #1000H
MOVX @DPTR, A
```

➤ MCS-51单片机读写片外数据存储器中的内容，除用MOVX A, @DPTR和MOVX @DPTR, A外，还可使用MOVX A, @Ri和MOVX @Ri, A。这时通过P0口输出Ri中的内容（低8位地址），而把P2口原有的内容作为高8位地址输出。

例：将程序存储器中以TAB为首址的32个单元的内容依次传送到外部RAM以7000H为首地址的区域去。

■ **DPTR**指向标号TAB的首地址。**R0**既指示外部RAM的地址，又表示数据标号TAB的位移量。本程序的循环次数为32，R0的值：0～31，R0的值达到32就结束循环。程序如下：

```
        MOV    P2, #70H
        MOV    DPTR, #TAB

        MOV    R0, #0
AGIN:   MOV    A, R0
        MOVC   A, @A+DPTR
        MOVX   @R0, A
        INC    R0
        CJNE   R0, #32, AGIN
HERE:   SJMP   HERE
TAB:    DB     .....
```



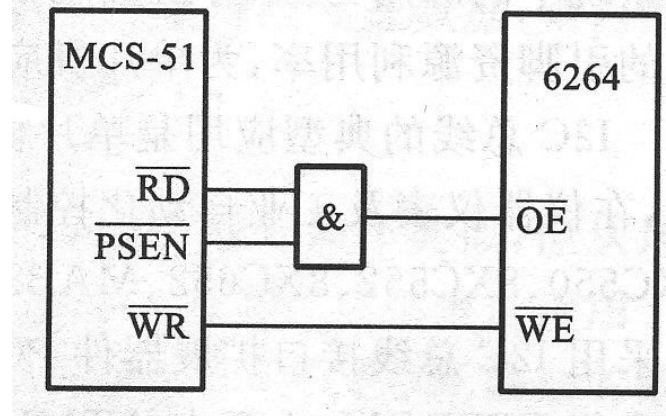
## 7.2 存储空间分配和片选信号产生方法

### 7.2.3 程序存储器和数据存储器的并行拓展

#### ➤ 数据空间和程序空间的合并

➤ ROM = 程序存储器? RAM = 数据存储器?

➤ 区分程序存储器和数据存储器的本质是什么?



## 7.2.3 程序存储器扩展

例：数据存储器与程序存储器并行拓展（P173 图7.18）

